Trường: _____

Khoa: _____

Môn học: _____

BÁO CÁO ĐỒ ÁN ỨNG DỤNG DI ĐỘNG

HKchat

Ứng dụng nhắn tin nội bộ cho nhóm làm việc

Nhóm HK

Phạm Đăng Khải · Trần Vũ Đình Huy

Tháng 11 năm 2026

HKchat kết hợp kênh thảo luận, trò chuyện trực tiếp và tìm kiếm tin nhắn trong một ứng dụng di động. Báo cáo trình bày phần mềm đã xây dựng, thiết kế giao diện, dữ liệu và kết quả kiểm thử có trong dự án.

Mục lục

Bảng phân công công việc 3
1. Giới thiệu chung và Nghiên cứu thị trường 4
2. Hình thành ý tưởng và lý do lựa chọn 7
3. Phân tích yêu cầu hệ thống 9
4. Câu chuyện người dùng và phạm vi MVP 12
5. Kiến trúc hệ thống và đặc tả kỹ thuật 18
6. Luồng vận hành hệ thống 22
7. Mô hình dữ liệu 26
8. Thiết kế giao diện và trải nghiệm 29
9. Triển khai và kiểm thử 36
10. Ràng buộc và giả định 40
11. Kết luận và hướng phát triển 41
Tài liệu tham khảo 43

Thông tin dự án

- **Ứng dụng.** HKchat.
- **Nhóm.** HK: Phạm Đăng Khải và Trần Vũ Đình Huy.
- **Client.** React Native cho Android; bản Expo Go để thử trên iPhone.
- **Server.** Node.js API và SQLite chạy trong Docker.
- **Thiết kế.** Figma với 18 trạng thái màn hình và trang Design System.

Bảng phân công công việc

Điền mã số sinh viên, nhiệm vụ thực tế và tỷ lệ hoàn thành của từng thành viên trước khi nộp.

STT	Họ và tên	MSSV	Công việc thực hiện	Tỷ lệ hoàn thành
1	Phạm Đăng Khải			
2	Trần Vũ Đình Huy			

1. Giới thiệu chung và Nghiên cứu thị trường

1.1 Bối cảnh và mục tiêu

Nhóm làm việc thường cần thông báo chung, trao đổi theo chủ đề và nhắn riêng giữa hai thành viên. HKchat đưa các cách trao đổi này vào cùng một ứng dụng di động. Người dùng có thể quay lại lịch sử hội thoại và tìm nội dung theo từ khóa.

Bản hiện tại tập trung vào giao tiếp bằng văn bản: tạo tài khoản, đăng nhập, kênh công khai, DM, hội thoại nhóm mẫu, gửi tin, trả lời, cảm xúc, sửa/xóa và tìm kiếm. Dữ liệu được lưu trên máy chủ local để nhóm vận hành và kiểm thử.

1.2 Khảo sát năm sản phẩm tương đồng

Phương pháp khảo sát là đọc tài liệu chức năng chính thức của năm sản phẩm nhắn tin cho nhóm và đối chiếu với các luồng đã triển khai trong HKchat. Đây là khảo sát tài liệu thứ cấp; nhóm chưa phỏng vấn người dùng hoặc đo hiệu năng của các sản phẩm. Vì vậy, "khoảng trống" dưới đây chỉ nêu khác biệt về phạm vi của bản HKchat, không kết luận sản phẩm khác kém chất lượng.

1. Slack. Đặc điểm và thế mạnh: Kênh và tin nhắn trực tiếp là hai cách tổ chức hội thoại chính; tìm kiếm hỗ trợ lọc theo cuộc trò chuyện và người gửi.

Giới hạn đối chiếu và lựa chọn cho HKchat: Nhiều tính năng cộng tác ngoài phạm vi đồ án. HKchat chọn luồng ngắn gồm kênh, DM, reply và tìm tin bằng từ khóa; chưa có bộ lọc tìm kiếm nâng cao.

Bài học áp dụng: Điểm rút ra cho thiết kế là phải làm rõ điểm vào của mỗi loại hội thoại. Danh sách kênh cần hiển thị ngay sau đăng nhập; DM cần mở từ danh sách người dùng; kết quả tìm kiếm phải quay lại hội thoại. Ba luồng này đã được đặt vào ba tab của client. Các bộ lọc theo người gửi hoặc thời gian có thể bổ sung sau khi luồng cơ bản được kiểm thử ổn định.

Nguồn tài liệu: https://slack.com/help/articles/1500000019301-Find-and-start-conversations

2. Microsoft Teams. Đặc điểm và thế mạnh: Nhóm và kênh gắn với dự án hoặc đơn vị, có phân loại kênh tiêu chuẩn, riêng tư và chia sẻ.

Giới hạn đối chiếu và lựa chọn cho HKchat: HKchat chưa có mô hình team, vai trò quản trị hay kênh riêng tư. Bản hiện tại dùng kênh công khai và membership để giới hạn quyền đọc hội thoại.

Bài học áp dụng: Khảo sát này cho thấy tên kênh và phạm vi thành viên là hai thông tin khác nhau. Trong HKchat, kênh công khai có thể xuất hiện ở danh sách tham gia, còn DM và nhóm không xuất hiện ở đó. Server kiểm tra channel_members trước khi đọc tin. Giải pháp này đáp ứng bài toán thử nghiệm nhóm nhỏ, nhưng chưa thay thế được mô hình quản trị tổ chức hoặc nhiều cấp quyền như Teams.

Nguồn tài liệu: https://learn.microsoft.com/en-us/microsoftteams/teams-channels-overview

3. Google Chat. Đặc điểm và thế mạnh: Space hỗ trợ trao đổi nhóm; trả lời một tin có thể mở luồng riêng để giữ ngữ cảnh.

Giới hạn đối chiếu và lựa chọn cho HKchat: HKchat lưu reply_to và hiển thị trích dẫn trong hội thoại. Bản hiện tại chưa có trải nghiệm thread tách riêng đầy đủ như sản phẩm tham chiếu.

Bài học áp dụng: Ý tưởng quan trọng là phản hồi phải gắn với tin gốc thay vì chỉ đứng sau nó theo thời gian. HKchat lưu id tin được trả lời trong messages.reply_to và API có endpoint đọc thread. Ở giao diện hiện tại, người dùng thấy phần trích dẫn khi trả lời; một màn hình thread độc lập và chỉ báo số phản hồi vẫn là hướng hoàn thiện. Vì vậy báo cáo tách khả năng của API khỏi mức độ hoàn thiện trên UI.

Nguồn tài liệu: https://support.google.com/chat/answer/12176589?hl=en

4. Zulip. Đặc điểm và thế mạnh: Tổ chức hội thoại bằng channel và topic; tìm kiếm có bộ lọc theo channel, topic, người gửi và DM.

Giới hạn đối chiếu và lựa chọn cho HKchat: HKchat chưa có topic. Tên kênh và liên kết reply_to giúp phân nhóm ở mức đồ án; tìm kiếm chỉ khớp từ khóa trong các kênh được phép đọc.

Bài học áp dụng: Zulip làm rõ lợi ích của việc gắn ngữ cảnh cho nội dung ngay từ lúc gửi. Với thời gian và phạm vi triển khai của đồ án, HKchat dùng tên kênh để tách chủ đề lớn và reply để nối câu trả lời với một tin cụ thể. Tìm kiếm theo từ khóa phục vụ việc tra lại; chưa có toán tử tìm kiếm hoặc topic. Đây là giới hạn chức năng được nêu rõ để không đánh đồng hai mô hình tổ chức hội thoại.

Nguồn tài liệu: https://zulip.com/help/getting-started-with-zulip

5. Rocket.Chat. Đặc điểm và thế mạnh: Có kênh, nhóm, DM, thread, tìm kiếm và lựa chọn tự vận hành máy chủ.

Giới hạn đối chiếu và lựa chọn cho HKchat: HKchat cũng chạy server riêng nhưng chỉ là bản phục vụ nhóm nhỏ; chưa có quản trị tổ chức, chia sẻ tệp hoặc thông báo đẩy như hệ sản phẩm đầy đủ.

Bài học áp dụng: Sản phẩm tham chiếu này cho thấy tự vận hành máy chủ là một lựa chọn phổ biến, không phải điểm độc quyền của HKchat. Đóng góp của đồ án nằm ở việc tự thiết kế và kiểm chứng một phiên bản nhỏ: Node.js API, SQLite, Docker volume, client React Native và bộ kiểm thử quyền truy cập. Báo cáo chỉ đánh giá những phần nhóm thực sự chạy được trên máy local và thiết bị thử.

Nguồn tài liệu: https://docs.rocket.chat/docs/our-plans

1.3 Bảng đối sánh và khoảng trống giải pháp

Tiêu chí	Sản phẩm tham chiếu	HKchat hiện tại	Căn cứ
Kênh / nhóm	Cả năm sản phẩm có cách trao đổi nhóm	Kênh công khai; nhóm mẫu 4 người	channels, group_chats, ảnh Android
DM	Slack, Teams, Google Chat, Zulip, Rocket.Chat	DM hai người, khóa cặp người dùng	POST /api/dm, dm_key
Trả lời theo ngữ cảnh	Slack/Google Chat/Rocket.Chat có thread	reply_to, trích dẫn; chưa có đầy đủ màn thread riêng	messages.reply_to, App.tsx
Tìm kiếm	Có ở các sản phẩm tham chiếu	Từ khóa trong kênh có quyền; tối đa 50 kết quả	GET /api/search
Vận hành server	Rocket.Chat có bản tự vận hành	Node.js, SQLite, Docker trên máy nhóm	compose.yaml

Cơ hội sản phẩm của HKchat trong phạm vi đồ án là cung cấp một bộ chức năng giao tiếp văn bản dễ trình diễn, có cơ sở dữ liệu riêng và đường chạy local rõ ràng. Đây là lựa chọn triển khai có chủ đích; báo cáo không khẳng định HKchat có ưu thế tính năng hoặc hiệu năng so với các nền tảng trưởng thành.

1.4 Mục tiêu và phạm vi đồ án

Từ việc đối chiếu năm sản phẩm, nhóm lựa chọn một vòng giao tiếp cốt lõi gồm kênh, tin nhắn trực tiếp, trả lời và tìm kiếm. Mục tiêu kỹ thuật là xây dựng client di động, API có xác thực và một database độc lập để mọi hành động trên có thể được kiểm tra bằng thao tác thật. Mục tiêu sản phẩm là giúp nhóm nhỏ trao đổi văn bản theo chủ đề trên server do nhóm vận hành.

1.5 Phạm vi sử dụng của bản hiện tại

HKchat giải quyết bài toán liên lạc bằng văn bản cho một nhóm có máy chủ do chính nhóm vận hành. Người dùng không phải chọn một dịch vụ chat công cộng bên ngoài: họ nhập URL của server HKchat, tạo tài khoản trên database local và làm việc với kênh hoặc DM của cùng server. Cách triển khai này phù hợp khi nhóm cần trình diễn chức năng trên một máy Windows và một số thiết bị trong cùng mạng thử nghiệm.

Một cuộc trao đổi thông thường bắt đầu từ danh sách kênh. Thành viên chọn General, Random hoặc kênh đã tham gia; tin được hiển thị theo thứ tự và làm mới định kỳ. Nếu cần trao đổi riêng, tab Mọi người mở DM hai người. Nếu cần tìm lại thông tin, tab Tìm kiếm trả nội dung theo từ khóa trong phạm vi quyền xem. Sửa, xóa, trả lời và cảm xúc là các thao tác sau khi cuộc trò chuyện đã được mở.

Bản dữ liệu mẫu có tài khoản và hội thoại bốn thành viên để báo cáo chức năng nhóm. Nội dung nói chuyện về đồ án chỉ là dữ liệu kiểm thử; đối tượng sản phẩm là nhóm làm việc nội bộ. Tên người dùng trong ảnh vì vậy không đại diện cho vai trò cố định trong doanh nghiệp. Dữ liệu thật có thể khác khi người dùng tự đăng ký và tạo kênh mới.

Phạm vi báo cáo dựa trên mã chạy, test server, ảnh Android và file Figma. Schema có thêm một số bảng dành cho các hướng mở rộng nhưng khi chưa có route hoặc màn hình tương ứng, báo cáo không liệt kê chúng là tính năng hiện hữu. Sự phân biệt này giúp hội đồng đối chiếu từng mô tả với nguồn minh chứng rõ ràng.

1.6 Các bên tham gia vào một cuộc trò chuyện

Người gửi chịu trách nhiệm nội dung mình viết. Trong API, user_id của tin được lấy từ phiên xác thực, không lấy từ dữ liệu JSON do client tự khai. Người đọc phải là thành viên kênh ở thời điểm gọi API. Người tạo kênh công khai được thêm vào membership ngay khi tạo; người khác có thể tham gia qua endpoint join. Trong DM, hai thành viên được thêm cùng lúc trong transaction.

Server là nơi quyết định quyền; giao diện giúp thao tác thuận tiện nhưng không phải lớp bảo vệ duy nhất. Ví dụ một nút Sửa có thể chỉ xuất hiện cho tin của mình, nhưng nếu người dùng gửi PATCH thủ công vào tin của người khác thì server vẫn trả 403. Tương tự, biết id một tin trong kênh khác không cho phép đọc thread hoặc gắn reaction.

Quá trình dùng app vì vậy có hai loại trạng thái cần theo dõi: trạng thái giao diện như tab đang chọn, kênh đang mở và bản nháp; trạng thái bền vững trên server như phiên, membership và các hàng tin nhắn. Khi khởi động lại app hoặc server, phần giao diện được dựng lại từ API và database thay vì giả định state cũ trong bộ nhớ còn tồn tại.

2. Hình thành ý tưởng và lý do lựa chọn

2.1 Phát biểu bài toán

Một nhóm làm việc cần có nơi để thông báo chung, trao đổi theo chủ đề, nói chuyện riêng và tìm lại quyết định cũ. Nếu toàn bộ nội dung nằm trong một luồng chat duy nhất, các chủ đề dễ trộn lẫn và việc tìm thông tin trở nên khó hơn. Nếu phải cấu hình nhiều dịch vụ cho một bản trình diễn nhỏ, nhóm gặp thêm công việc vận hành. Bài toán của HKchat là tạo một hệ thống di động có kênh, DM, liên kết trả lời và tìm kiếm, chạy được trên server do nhóm quản lý.

2.2 Đối tượng và bối cảnh sử dụng

- **Thành viên mới.** *Nhu cầu:* Tạo tài khoản, vào kênh chung. *Tình huống tiêu biểu:* Nhập URL server, đăng ký, thấy General và Random.
- **Thành viên đang làm việc.** *Nhu cầu:* Trao đổi chung và riêng. *Tình huống tiêu biểu:* Mở kênh theo chủ đề hoặc DM một đồng nghiệp.
- **Người cần tra cứu.** *Nhu cầu:* Tìm lại một tin và ngữ cảnh. *Tình huống tiêu biểu:* Gõ từ khóa, chọn kết quả, trở lại hội thoại.
- **Người vận hành.** *Nhu cầu:* Khởi động API và giữ dữ liệu. *Tình huống tiêu biểu:* Docker Compose, healthcheck, SQLite volume.

2.3 Giá trị cốt lõi và phạm vi khả thi

Giá trị cốt lõi của bản HKchat là tính liên tục của một vòng trao đổi: vào đúng server, đăng nhập, chọn đúng cuộc trò chuyện, gửi tin, trả lời và tìm lại. Vì vậy, nhóm ưu tiên chức năng có thể nối thành một hành trình sử dụng hoàn chỉnh. Kênh và DM dùng chung mô hình messages nhưng có điều kiện quyền khác nhau. Đây là quyết định kỹ thuật để giảm sự trùng lặp logic và giúp kiểm thử quyền truy cập theo cùng một chuẩn.

Trong bản đang chạy, các thao tác văn bản và tìm kiếm đã được triển khai. Hội thoại nhóm bốn người được chuẩn bị sẵn làm dữ liệu trình diễn; API có thể tạo nhóm nhưng giao diện chưa có nút tạo nhóm. Push notification, tải tệp và cuộc gọi chưa được xem là chức năng của bản này. Cách giới hạn này giúp tiêu chí nghiệm thu dựa trên thứ có thể quan sát được thay vì những ý tưởng chưa có mã hoặc bằng chứng.

2.4 Mục tiêu có thể kiểm chứng

- **Tài khoản và phiên.** *Cách kiểm chứng:* Đăng ký, đăng nhập, đăng xuất, thử mật khẩu sai. *Minh chứng trong dự án:* API test và màn hình xác thực.
- **Trao đổi có quyền.** *Cách kiểm chứng:* Thử thành viên và người ngoài đọc DM/kênh. *Minh chứng trong dự án:* server.test.mjs, HTTP 403.
- **Giữ ngữ cảnh.** *Cách kiểm chứng:* Gửi reply, đọc trích dẫn và mở thread. *Minh chứng trong dự án:* reply_to, API, ảnh hội thoại.
- **Tra cứu.** *Cách kiểm chứng:* Tìm từ khóa rồi mở hội thoại. *Minh chứng trong dự án:* GET /api/search, ảnh Android.
- **Vận hành lặp lại.** *Cách kiểm chứng:* Khởi động Docker, gọi healthcheck. *Minh chứng trong dự án:* compose.yaml, README.

2.5 Business Model Canvas cho bản đồ án

Canvas mô tả cách HKchat phục vụ nhóm dùng thử trong phạm vi dự án hiện tại. Sản phẩm được vận hành trên server local và không thu phí người dùng.

Khối	Nội dung của HKchat
Phân khúc người dùng	Nhóm nhỏ và bộ phận nội bộ cần trao đổi theo kênh, DM.
Giá trị cung cấp	Một giao diện di động cho nhắn tin, reply và tìm lại nội dung trên server riêng.
Kênh tiếp cận	Bản demo Android, Figma prototype và hướng dẫn chạy trong README.
Quan hệ người dùng	Thành viên tự đăng ký tài khoản trên máy chủ của nhóm.
Dòng doanh thu	Không thu phí trong bản đồ án đang vận hành.
Nguồn lực chính	Mã React Native, Node.js API, SQLite, Docker và thiết kế Figma.
Hoạt động chính	Vận hành API, quản lý dữ liệu và kiểm thử các luồng giao tiếp.
Đối tác chính	Hai thành viên nhóm HK phụ trách thiết kế và phát triển dự án.
Cơ cấu chi phí	Thời gian phát triển và tài nguyên máy dùng chạy Docker, Android Emulator.

3. Phân tích yêu cầu hệ thống

3.1 Chức năng đã triển khai

Ở tầng tài khoản, HKchat cho phép người dùng chọn địa chỉ server, đăng ký, đăng nhập bằng tên hoặc email, khôi phục phiên và đăng xuất. Sau khi vào ứng dụng, người dùng xem kênh đã tham gia, tạo hoặc tham gia kênh công khai, mở DM từ danh sách thành viên và đọc hội thoại nhóm mẫu. Trong hội thoại, người dùng gửi tin, trả lời, sửa hoặc xóa tin của mình và thả cảm xúc. Tab Tìm kiếm cho phép tìm từ khóa trong phạm vi hội thoại được phép đọc rồi mở lại cuộc trò chuyện. Các nhóm chức năng này được mô tả theo điều kiện đầu vào và phản hồi ở các mục sau.

3.2 Ma trận tác động và công sức

Nhóm đánh giá tác động lên luồng giao tiếp và công sức triển khai tương đối. Các mục trong bảng đều có trong bản ứng dụng hiện tại.

Chức năng	Tác động	Công sức	Lý do
Tài khoản và phiên	Cao	Vừa	Xác định người dùng và quyền truy cập.
Kênh, DM, gửi tin	Cao	Vừa	Hoàn thành giao tiếp hằng ngày.
Reply và tìm kiếm	Cao	Vừa	Giữ ngữ cảnh và truy xuất nội dung.
Sửa, xóa, cảm xúc	Vừa	Thấp	Hoàn thiện thao tác với tin.

3.3 Đặc tả yêu cầu chức năng

Nhóm yêu cầu tài khoản bao gồm đăng ký, đăng nhập, khôi phục phiên khi mở lại app và đăng xuất. Tên đăng nhập được server chấp nhận khi có 3–32 ký tự chữ, số hoặc dấu gạch dưới; email phải có định dạng hợp lệ; mật khẩu tối thiểu 8 ký tự. API trả HTTP 409 nếu tên hoặc email đã tồn tại. Người dùng nhập địa chỉ server trước khi xác thực; client kiểm tra URL có dạng http(s)://host và lưu địa chỉ dùng gần nhất.

Sau khi đăng ký, server tạo hoặc tìm hai kênh công khai General và Random rồi thêm tài khoản mới vào bảng thành viên. Điều này giúp tài khoản mới có nội dung điều hướng ngay khi vào ứng dụng. Khi đăng nhập, tên hoặc email đều được chấp nhận làm định danh. Phiên được cấp token ngẫu nhiên và lưu dạng hash kèm thời điểm hết hạn sau 30 ngày; thao tác Đăng xuất xóa token trên server và thông tin phiên trong app.

Nhóm yêu cầu kênh bao gồm xem kênh đã tham gia, xem kênh công khai khác, tạo kênh và tham gia kênh. Tên kênh mới phải dài 2–80 ký tự. API chỉ cho phép tham gia kênh công khai không phải group chat. Giao diện hiển thị tên, biểu tượng và hành động Tham gia. Nếu kênh đã mở, màn hội thoại đọc tin theo quyền thành viên.

Tin nhắn trực tiếp được mở từ tab Mọi người. Client gửi userId của người nhận; server xác nhận tài khoản tồn tại và không phải chính người gửi. Khóa dm_key ghép hai userId theo thứ tự ổn định để cùng một cặp người chỉ có một cuộc DM. Bảng channel_members chứa đúng hai thành viên khi tạo DM. Người thứ ba không thể đọc hoặc tìm nội dung cuộc trò chuyện này.

Nhóm hội thoại được API tạo với một người tạo và 2–19 thành viên khác, tổng 3–20 người. Server xác nhận mọi userId, loại bỏ khả năng trùng hoặc tự thêm chính mình. Bản dữ liệu mẫu có cuộc trò chuyện bốn người để kiểm thử giao diện; client hiện hiển thị nhóm này như một cuộc hội thoại trong danh sách kênh. Báo cáo không mô tả màn hình tự tạo nhóm vì client chưa có thao tác đó.

- **FR01.** *Đầu vào và điều kiện:* Tên 3–32 ký tự, email hợp lệ, mật khẩu ≥8. *Kết quả hệ thống:* Tạo users, phiên và gắn General/Random.
- **FR02.** *Đầu vào và điều kiện:* Tên hoặc email và mật khẩu đúng. *Kết quả hệ thống:* Cấp phiên, mở danh sách kênh.
- **FR03.** *Đầu vào và điều kiện:* Tên kênh 2–80 ký tự. *Kết quả hệ thống:* Tạo kênh công khai và thêm người tạo.
- **FR04.** *Đầu vào và điều kiện:* Kênh công khai có thật. *Kết quả hệ thống:* Thêm người dùng vào channel_members.
- **FR05.** *Đầu vào và điều kiện:* Người nhận DM hợp lệ. *Kết quả hệ thống:* Tạo hoặc dùng lại DM của cùng cặp.
- **FR06.** *Đầu vào và điều kiện:* Nhóm 3–20 người, danh sách duy nhất. *Kết quả hệ thống:* Tạo group_chats và thành viên.

3.4 Đặc tả thao tác với tin nhắn

Người dùng chỉ gửi tin khi thuộc kênh. Server cắt khoảng trắng đầu/cuối và từ chối nội dung trống hoặc dài hơn 4000 ký tự. Nếu tin là phản hồi, replyTo phải trỏ đến một tin chưa xóa trong cùng kênh. Tin được gán id tăng tự động, user_id tác giả và thời điểm tạo. Client làm mới hội thoại mỗi hai giây khi đang mở để lấy nội dung mới từ server.

Danh sách tin ban đầu lấy tối đa 100 tin mới nhất, sau đó trả về theo thứ tự cũ đến mới. API cũng hỗ trợ tham số after để lấy tối đa 100 tin có id lớn hơn mốc đã biết. Cách này giúp client có khả năng truy vấn bổ sung mà không đọc toàn bộ lịch sử. Tin có deleted_at không được trả trong danh sách thông thường.

Tác giả có thể sửa nội dung bằng PATCH và xóa bằng DELETE. Khi sửa, server kiểm tra nội dung theo cùng giới hạn 1–4000 ký tự và ghi edited_at. Xóa là xóa mềm: server ghi deleted_at, giữ bản ghi để liên kết reply không bị hỏng. Người khác trong kênh vẫn đọc được tin trước khi xóa nhưng không thể sửa hoặc xóa thay tác giả.

Cảm xúc là bản ghi riêng theo bộ ba message_id, user_id và emoji. Trùng một cảm xúc của cùng người trên cùng tin không tạo thêm hàng; thao tác xóa bỏ đúng hàng của người đó. API tổng hợp số lượt và cờ me cho từng emoji khi trả tin. Giao diện cung cấp bốn biểu tượng 👍, ❤️, 😂 và 🎉, cùng nút mở bộ chọn cảm xúc bên dưới mỗi tin.

Tìm kiếm nhận một từ khóa tối đa 100 ký tự; chuỗi rỗng trả danh sách trống. Truy vấn dùng bảng thành viên để giới hạn kết quả trong các kênh người dùng được phép xem, bỏ tin đã xóa và trả tối đa 50 kết quả mới nhất. Giao diện hiển thị tác giả và nội dung; chạm vào kết quả mở cuộc hội thoại tương ứng.

- **FR07.** *Ràng buộc:* Tin 1–4000 ký tự, người gửi là thành viên. *Phản hồi lỗi hoặc trạng thái:* 400 nếu nội dung sai; 403 nếu ngoài kênh.
- **FR08.** *Ràng buộc:* replyTo là tin còn tồn tại trong cùng kênh. *Phản hồi lỗi hoặc trạng thái:* 400 nếu tin đích sai.
- **FR09.** *Ràng buộc:* Sửa/xóa chỉ dành cho tác giả. *Phản hồi lỗi hoặc trạng thái:* 403 nếu người khác thao tác.
- **FR10.** *Ràng buộc:* Emoji không rỗng, tối đa 64 ký tự. *Phản hồi lỗi hoặc trạng thái:* 400 nếu emoji không hợp lệ.
- **FR11.** *Ràng buộc:* Từ khóa tối đa 100 ký tự. *Phản hồi lỗi hoặc trạng thái:* Trả tối đa 50 tin trong kênh có quyền.

3.5 Yêu cầu phi chức năng được kiểm tra

Bảo vệ dữ liệu bắt đầu ở xác thực: mật khẩu được xử lý bằng scrypt với salt ngẫu nhiên riêng trước khi ghi vào users; token phiên được băm SHA-256 trước khi lưu vào sessions. Server so sánh hash mật khẩu bằng timingSafeEqual. Các endpoint ngoài đăng ký, đăng nhập và health đều kiểm tra Bearer token còn hạn. Kiểm tra quyền thành viên được đặt trước thao tác đọc và gửi tin, mở thread hoặc thả cảm xúc.

Tính toàn vẹn dữ liệu dựa trên khóa ngoại SQLite, CHECK cho kind và độ dài body, UNIQUE cho username/email/dm_key và các khóa chính ghép trong bảng thành viên, cảm xúc. Các thao tác tạo tài khoản, kênh, nhóm và DM dùng transaction để tránh trạng thái chỉ tạo một phần khi có lỗi. Chế độ WAL được bật khi mở database để hỗ trợ ghi và đọc ổn định trong phạm vi server local.

Khả năng cài đặt lặp lại được thể hiện qua Docker Compose, volume giữ database và README từng bước cho Windows. Người dùng Android Emulator dùng 10.0.2.2 để vào máy host; iPhone chạy Expo Go dùng IP LAN. Đây là hai đường kiểm thử đã mô tả trong dự án. Báo cáo không đặt chỉ tiêu thời gian phản hồi, FPS hoặc tải đồng thời vì nhóm chưa có phép đo tương ứng.

Khả năng sử dụng được hỗ trợ bởi nhãn rõ ràng cho ô nhập và nút, lỗi xác thực hiển thị trong màn đăng nhập, thanh ngữ cảnh khi reply/sửa, phản hồi khi thao tác đang bận và ba tab cố định sau đăng nhập. Những quyết định này có thể quan sát trong mã React Native và ảnh Android; đánh giá khả dụng định lượng cần nghiên cứu người dùng riêng.

4. Câu chuyện người dùng và phạm vi MVP

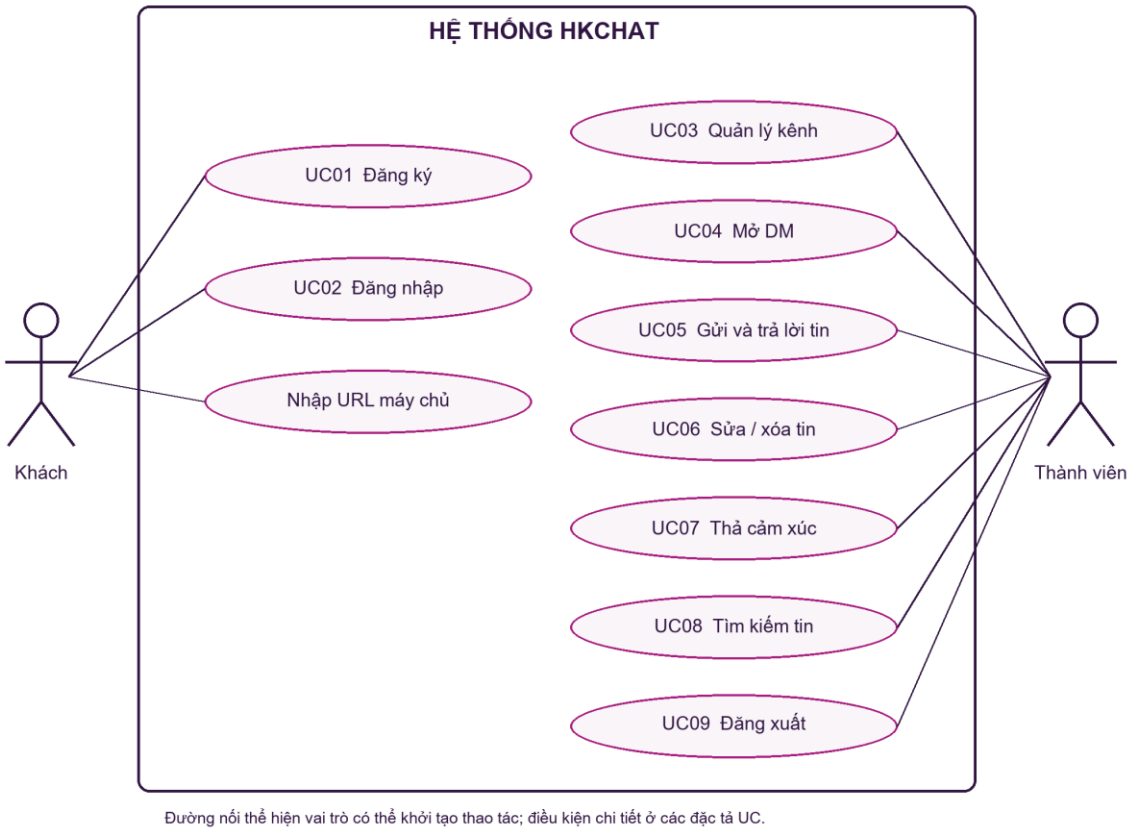
4.1 Tiêu chí nghiệm thu theo hành trình người dùng

- **US01.** *Câu chuyện người dùng:* Là người mới, tôi muốn đăng ký để tham gia nhóm. *Tiêu chí nghiệm thu:* Tên 3–32 ký tự, email hợp lệ, mật khẩu ≥8; dữ liệu đúng tạo user và phiên; trùng trả 409.
- **US02.** *Câu chuyện người dùng:* Là thành viên, tôi muốn đăng nhập để xem cuộc trò chuyện của mình. *Tiêu chí nghiệm thu:* Tên hoặc email và mật khẩu đúng trả token; sai trả 401; client tải lại danh sách kênh.
- **US03.** *Câu chuyện người dùng:* Tôi muốn tạo kênh công khai để trao đổi theo chủ đề. *Tiêu chí nghiệm thu:* Tên 2–80 ký tự; người tạo trở thành thành viên; kênh hiện trong danh sách.
- **US04.** *Câu chuyện người dùng:* Tôi muốn tham gia kênh công khai đã có. *Tiêu chí nghiệm thu:* Kênh công khai chưa tham gia xuất hiện riêng; bấm Tham gia thêm membership.
- **US05.** *Câu chuyện người dùng:* Tôi muốn nhắn riêng cho một người. *Tiêu chí nghiệm thu:* Cùng cặp tài khoản mở lại cùng DM; người thứ ba nhận 403 khi đọc.
- **US06.** *Câu chuyện người dùng:* Tôi muốn gửi và xem tin trong hội thoại. *Tiêu chí nghiệm thu:* Tin 1–4000 ký tự được lưu; danh sách trả tối đa 100 tin; người ngoài bị từ chối.
- **US07.** *Câu chuyện người dùng:* Tôi muốn trả lời một tin cụ thể. *Tiêu chí nghiệm thu:* replyTo phải là tin còn tồn tại cùng kênh; tin mới hiển thị trích dẫn.
- **US08.** *Câu chuyện người dùng:* Tôi muốn sửa hoặc xóa tin do mình viết. *Tiêu chí nghiệm thu:* Tác giả thao tác được; người khác nhận 403; tin xóa không xuất hiện trong danh sách.
- **US09.** *Câu chuyện người dùng:* Tôi muốn thể hiện cảm xúc với tin. *Tiêu chí nghiệm thu:* Cùng người và emoji không tạo trùng; bỏ cảm xúc cập nhật tổng số.
- **US10.** *Câu chuyện người dùng:* Tôi muốn tìm và mở lại nội dung. *Tiêu chí nghiệm thu:* Chỉ trả tin trong kênh có quyền; chạm kết quả mở đúng cuộc trò chuyện.

Các tiêu chí trên được viết theo trạng thái hiện có của mã nguồn và bài test. Một số tiêu chí về giao diện được đối chiếu qua ảnh Android và prototype Figma; chúng không thay thế cho kiểm thử định lượng với người dùng thật.

4.2 Sơ đồ và đặc tả use case

Sơ đồ dưới đây chỉ mô tả những thao tác mà người dùng có thể bắt đầu từ giao diện HKchat hiện tại. Nhóm hội thoại đã có được đọc và gửi tin theo cùng luồng kênh; API tạo nhóm chưa có màn hình tạo nhóm nên không đặt thành use case của người dùng di động. Mỗi đặc tả ghi tác nhân, tiền điều kiện, luồng chính, nhánh lỗi và kết quả để nối với user stories và ca kiểm thử ở các phần sau.



Hình UC-01. Sơ đồ use case của bản HKchat đang chạy

UC01 — Đăng ký tài khoản

Tác nhân và mục tiêu: Khách. Tạo tài khoản trên server HKchat để tham gia trò chuyện.

Tiền điều kiện: Server hoạt động; người dùng nhập URL hợp lệ và chưa có phiên đăng nhập.

Luồng chính:

1. Chọn Đăng ký rồi nhập tên đăng nhập, email và mật khẩu.

2. Client gửi POST /api/register tới URL đã chọn.

3. Server kiểm tra tên 3–32 ký tự, email hợp lệ, mật khẩu ít nhất 8 ký tự và tính duy nhất của tên/email; tạo user, gắn General/Random và cấp phiên.

4. Client lưu token và URL, tải danh sách kênh và mở màn chính.

Luồng thay thế — Dữ liệu sai: Server trả 400; form hiển thị lỗi để sửa và gửi lại.

Luồng thay thế — Tên/email trùng: Server trả 409; không tạo tài khoản thứ hai.

Luồng thay thế — Không kết nối được: Form giữ người dùng ở màn xác thực và báo lỗi kết nối.

Hậu điều kiện: Tài khoản tồn tại trong users, phiên trong sessions và người dùng là thành viên General/Random.

Minh chứng/điểm kiểm tra: FR01, US01; màn Figma 02–03; App.tsx authenticate(); POST /api/register; test đăng ký hợp lệ.

UC02 — Đăng nhập và khôi phục phiên

Tác nhân và mục tiêu: Khách hoặc thành viên quay lại. Vào đúng dữ liệu của tài khoản.

Tiền điều kiện: Server có tài khoản; URL server hợp lệ.

Luồng chính:

1. Nhập tên đăng nhập hoặc email cùng mật khẩu và bấm Đăng nhập.

2. Client gửi POST /api/login; server đối chiếu mật khẩu và cấp Bearer token có hạn.

3. Client lưu token trong Keychain, gọi API lấy kênh rồi hiển thị ba tab chính.

4. Khi mở lại ứng dụng, client đọc token đã lưu, gọi GET /api/me và tải lại kênh nếu phiên còn hợp lệ.

Luồng thay thế — Sai thông tin: Server trả 401; form giữ nguyên để nhập lại.

Luồng thay thế — Phiên cũ hết hạn: GET /api/me không hợp lệ; client trở về form xác thực.

Luồng thay thế — URL không đúng: Client báo định dạng URL sai hoặc lỗi kết nối trước khi vào ứng dụng.

Hậu điều kiện: Ứng dụng gắn với đúng user và chỉ dùng token còn hiệu lực.

Minh chứng/điểm kiểm tra: FR02, US02–US03; màn Figma 01/03; App.tsx authenticate()/khôi phục phiên; /api/login, /api/me.

UC03 — Xem, tạo và tham gia kênh

Tác nhân và mục tiêu: Thành viên đã đăng nhập. Trao đổi chung theo chủ đề.

Tiền điều kiện: Token còn hạn; API truy cập được.

Luồng chính:

1. Mở tab Kênh; client lấy kênh đã tham gia và kênh công khai còn có thể tham gia.

2. Chọn kênh đã có để đọc tin, hoặc nhập tên 2–80 ký tự và bấm Tạo kênh.

3. Khi tạo kênh, server thêm người tạo vào channel_members; khi chọn Tham gia, server thêm membership vào kênh public.

4. Client tải lại danh sách và mở hội thoại của kênh vừa chọn.

Luồng thay thế — Tên sai: POST /api/channels trả 400, kênh không được tạo.

Luồng thay thế — Kênh không công khai hoặc không tồn tại: Thao tác tham gia trả 404; client báo lỗi.

Luồng thay thế — Đọc trước khi tham gia: API tin nhắn trả 403; không lộ nội dung.

Hậu điều kiện: Kênh mới hoặc đã tham gia hiện trong danh sách của user; membership được lưu.

Minh chứng/điểm kiểm tra: FR03–FR04, US04–US05; màn Figma 04–06; GET/POST /api/channels, /api/channels/public, /join; server.test.mjs.

UC04 — Mở trò chuyện trực tiếp

Tác nhân và mục tiêu: Thành viên đã đăng nhập. Trao đổi riêng với một người khác.

Tiền điều kiện: Tài khoản người nhận tồn tại và khác tài khoản hiện tại.

Luồng chính:

1. Mở tab Mọi người và chọn người muốn nhắn.

2. Client gửi POST /api/dm với userId người nhận.

3. Server tính dm_key từ hai id; dùng lại DM đã có hoặc tạo channel mới và thêm đúng hai thành viên.

4. Client mở hội thoại DM để đọc và gửi tin.

Luồng thay thế — Người nhận sai: Server trả 400; không tạo DM.

Luồng thay thế — Người thứ ba truy cập: API đọc tin hoặc tìm kiếm không trả nội dung DM cho người ngoài.

Hậu điều kiện: Hai thành viên nhìn thấy cùng một DM, người ngoài không có quyền đọc.

Minh chứng/điểm kiểm tra: FR05, US06; màn Figma 07–08; POST /api/dm, channel_members; bài test DM và 403.

UC05 — Gửi và trả lời tin

Tác nhân và mục tiêu: Thành viên của cuộc trò chuyện. Gửi nội dung và giữ ngữ cảnh phản hồi.

Tiền điều kiện: Đã mở kênh, DM hoặc nhóm có membership; token còn hạn.

Luồng chính:

1. Nhập nội dung 1–4000 ký tự rồi bấm Gửi; nếu cần trả lời, chọn Trả lời trên tin gốc trước khi nhập.

2. Client gửi POST /api/channels/:id/messages với body và replyTo khi có tin gốc.

3. Server kiểm tra membership, nội dung và tin replyTo chưa bị xóa trong cùng kênh; lưu messages.

4. Client tải lại hội thoại; tin mới và trích dẫn phản hồi xuất hiện.

Luồng thay thế — Tin rỗng hoặc quá dài: Server trả 400; không ghi nội dung sai.

Luồng thay thế — replyTo sai kênh/không tồn tại: Server trả 400; không ghi phản hồi.

Luồng thay thế — Người ngoài kênh: Server trả 403 trước khi ghi tin.

Hậu điều kiện: Tin có tác giả, thời điểm và liên kết reply_to nếu là phản hồi.

Minh chứng/điểm kiểm tra: FR07–FR08, US07–US08; màn Figma 09–10; messages, reply_to; test gửi, reply và thread.

UC06 — Sửa hoặc xóa tin của mình

Tác nhân và mục tiêu: Tác giả tin. Điều chỉnh hoặc gỡ nội dung đã gửi.

Tiền điều kiện: Tin chưa bị xóa; người dùng còn là thành viên kênh và là tác giả.

Luồng chính:

1. Nhấn giữ tin của mình và chọn Sửa hoặc Xóa.

2. Với Sửa, client gửi PATCH /api/messages/:id cùng nội dung mới; server ghi body và edited_at.

3. Với Xóa, client hỏi xác nhận rồi gửi DELETE /api/messages/:id; server ghi deleted_at.

4. Client tải lại hội thoại để phản ánh nội dung mới hoặc bỏ tin đã xóa.

Luồng thay thế — Không phải tác giả: Server trả 403, tin không thay đổi.

Luồng thay thế — Tin đã xóa/không tồn tại: Server trả 404.

Luồng thay thế — Nội dung sửa rỗng: Server trả 400 và giữ nội dung cũ.

Hậu điều kiện: Tin sửa có edited_at; tin xóa mềm không còn ở danh sách và kết quả tìm kiếm.

Minh chứng/điểm kiểm tra: FR09, US09–US10; màn Figma 11–12/14; PATCH, DELETE /api/messages/:id; test quyền tác giả.

UC07 — Thêm hoặc bỏ cảm xúc

Tác nhân và mục tiêu: Thành viên kênh. Phản hồi nhanh đối với một tin.

Tiền điều kiện: Tin còn tồn tại và người dùng có quyền đọc kênh.

Luồng chính:

1. Chạm bộ chọn cảm xúc dưới tin và chọn một emoji.

2. Client gửi POST /api/messages/:id/reactions; server ghi bộ ba message_id, user_id, emoji nếu chưa có.

3. API trả tin với tổng count và cờ me; client hiển thị trạng thái mới.

4. Nếu bỏ cảm xúc, client gọi DELETE /api/messages/:id/reactions/:emoji và cập nhật count.

Luồng thay thế — Emoji không hợp lệ: Server trả 400.

Luồng thay thế — Người ngoài kênh: Server trả 403.

Luồng thay thế — Chọn lại cùng emoji: Khóa duy nhất giữ một reaction cho mỗi người/emoji/tin; count không tăng trùng.

Hậu điều kiện: Tổng cảm xúc của tin phản ánh các bản ghi hợp lệ trong reactions.

Minh chứng/điểm kiểm tra: FR10, US11; màn Figma 13; reactions; server.test.mjs kiểm tra count/me.

UC08 — Tìm và mở lại tin

Tác nhân và mục tiêu: Thành viên đã đăng nhập. Tìm lại nội dung có quyền xem.

Tiền điều kiện: Token còn hạn; người dùng đang ở tab Tìm kiếm.

Luồng chính:

1. Nhập từ khóa và bấm Tìm kiếm.

2. Client gọi GET /api/search?q=...; server tìm trong tin chưa xóa thuộc kênh người dùng tham gia.

3. API trả tối đa 50 kết quả mới nhất; client hiện tác giả và nội dung.

4. Chọn một kết quả để mở cuộc hội thoại chứa tin.

Luồng thay thế — Từ khóa rỗng: API trả danh sách rỗng, không tìm toàn bộ database.

Luồng thay thế — Không có kết quả: Danh sách kết quả trống; người dùng có thể nhập từ khóa khác.

Luồng thay thế — Tin của DM/kênh ngoài quyền: Không xuất hiện trong kết quả.

Hậu điều kiện: Đúng hội thoại được mở; bản hiện tại chưa tự cuộn tới vị trí của tin được chọn.

Minh chứng/điểm kiểm tra: FR11, US12; màn Figma 15–17; GET /api/search; test tìm kiếm theo membership.

UC09 — Đăng xuất

Tác nhân và mục tiêu: Thành viên đã đăng nhập. Kết thúc phiên trên thiết bị.

Tiền điều kiện: Client có token phiên đang dùng.

Luồng chính:

1. Bấm Đăng xuất và xác nhận trong hộp thoại.

2. Client gọi POST /api/logout; server xóa hash token của phiên hiện tại.

3. Client xóa token local, dữ liệu hội thoại đang mở và trở về màn xác thực.

Luồng thay thế — Hủy xác nhận: Không gọi API, giữ nguyên phiên.

Luồng thay thế — API logout lỗi mạng: Client vẫn xóa token trên thiết bị; phiên phía server hết hạn theo thời hạn đã cấp nếu request không tới được.

Hậu điều kiện: Thiết bị không tự vào lại app bằng token vừa xóa.

Minh chứng/điểm kiểm tra: US13; màn Figma 18; App.tsx logout(); POST /api/logout, Keychain.

4.3 Danh mục câu chuyện người dùng chi tiết

Câu chuyện người dùng được viết theo mục tiêu thao tác, điều kiện nghiệm thu và chứng cứ có thể kiểm tra. Một luồng được xem là hoàn thành khi API trả dữ liệu đúng và giao diện có đường đi để người dùng thực hiện, hoặc khi báo cáo ghi rõ trường hợp chỉ có API. Phần này không gán những yêu cầu chưa có trong app vào phạm vi đã triển khai.

Với tài khoản, điều kiện quan trọng là tên/email không trùng, mật khẩu đủ dài, phiên còn hiệu lực và token bị hủy khi đăng xuất. Với hội thoại, điều kiện quan trọng là membership chặn người ngoài, tác giả mới được sửa/xóa và reply không trỏ sang kênh khác. Với tìm kiếm, quyền xem phải được áp dụng trong truy vấn trước khi trả kết quả.

4.4 Liên kết yêu cầu với màn hình và API

Ma trận truy vết cho thấy mỗi yêu cầu chính có nơi thực hiện trên client và endpoint tương ứng. Nó đặc biệt hữu ích với đồ án có prototype Figma: số màn hình được nối không tự chứng minh server xử lý đúng. Ngược lại, một endpoint có test thành công cũng không tự chứng minh người dùng thao tác được trên giao diện.

Ví dụ, API /api/groups có test và dữ liệu nhóm mẫu, nhưng giao diện hiện không có form tạo nhóm. Vì vậy chức năng nhóm trong báo cáo được mô tả là đọc/nhắn trong nhóm đã có, không ghi thành luồng người dùng tự tạo nhóm. Việc chỉ ra ranh giới này giúp demo trùng với báo cáo khi giảng viên hỏi trực tiếp.

Yêu cầu	Use case	Màn hình	Thành phần/API
US01–US03	UC01–UC02	Figma 01–03	App.tsx xác thực; register/login/me; users, sessions
US04–US05	UC03	Figma 04–06	App.tsx tab Kênh; channels/public/join; channel_members
US06	UC04	Figma 07–08	App.tsx tab Mọi người; users/dm; dm_key
US07–US08	UC05	Figma 09–10	App.tsx hội thoại; messages/thread; reply_to
US09–US11	UC06–UC07	Figma 11–14	App.tsx thao tác tin; PATCH/DELETE; reactions
US12	UC08	Figma 15–17	App.tsx Tìm kiếm; GET /api/search; membership
US13	UC09	Figma 18	App.tsx logout; POST /api/logout; sessions

5. Kiến trúc hệ thống và đặc tả kỹ thuật

5.1 Cấu trúc hệ thống

Client React Native gọi HTTP API tới máy chủ Node.js. API kiểm tra phiên, quyền thành viên và dữ liệu đầu vào rồi đọc/ghi SQLite. Docker Compose chạy API trên cổng 8070 và gắn volume cho cơ sở dữ liệu. Các migration SQL được áp dụng theo thứ tự phiên bản khi server khởi động.

5.2 Mô hình dữ liệu

Mật khẩu được dẫn xuất bằng scrypt với salt riêng; database lưu giá trị băm. Token phiên cũng được băm khi lưu. Truy vấn đọc tin và tìm kiếm chỉ trả nội dung thuộc kênh mà người dùng là thành viên.

5.3 API chính

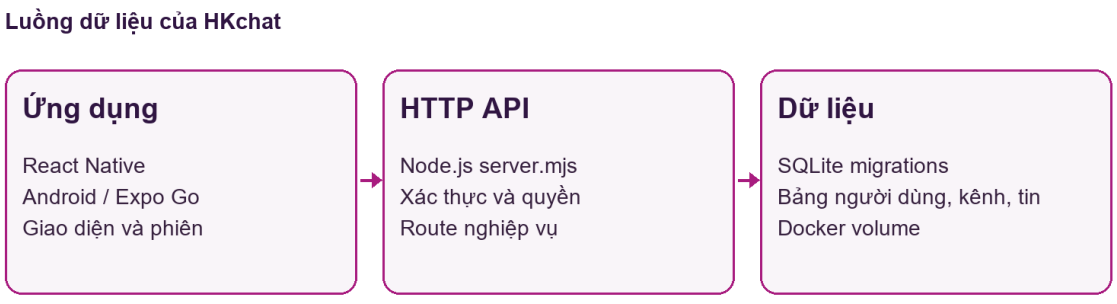
5.4 Cấu trúc mã nguồn và trách nhiệm từng thành phần

Tệp hkchat/mobile/App.tsx chứa client React Native. Phần đầu định nghĩa kiểu User, Channel, Message và hàm api() chung. Các state của component quản lý phiên đăng nhập, tab hiện tại, kênh đang mở, tin đã tải, bản nháp, tin đang trả lời hoặc sửa và danh sách kết quả tìm kiếm. Tách các trạng thái này giúp màn hình đổi giữa xác thực, danh sách và hội thoại mà vẫn giữ dữ liệu cần thiết cho mỗi thao tác.

Tệp hkchat/server/server.mjs khởi tạo database, áp dụng migration, định nghĩa helper cho mật khẩu và phiên, sau đó điều phối các route trong một HTTP handler. Các route công khai là health, register và login. Phần còn lại đi qua kiểm tra Bearer token trước khi xử lý. Việc đọc hoặc sửa tin còn có một lớp kiểm tra thành viên kênh và, với sửa/xóa, kiểm tra tác giả.

Ba migration SQL trong hkchat/server/migrations được áp dụng theo số phiên bản. 001_core.sql định nghĩa dữ liệu cốt lõi, 002_features.sql tạo thêm các bảng nền cho những phân hệ khác, còn 003_groups.sql bổ sung group_chats. Phần báo cáo chức năng chỉ căn cứ vào route và giao diện đã có; sự tồn tại của một bảng trong migration không đồng nghĩa tính năng tương ứng đã được triển khai ở client.

Tệp hkchat/compose.yaml mô tả service api dùng Node.js 22 Alpine, mount thư mục server ở chế độ chỉ đọc, gắn volume hkchat_data vào /data và công bố cổng 8070. Healthcheck gọi /api/health bên trong container. Khi volume còn trống, command có thể chép database mẫu từ thư mục backup; dữ liệu tạo sau đó nằm trong volume và tồn tại qua các lần khởi động lại container.



Hình. Sơ đồ các tầng client, API và SQLite

5.5 Đường đi của một request

Ví dụ khi gửi tin, client gọi POST /api/channels/:id/messages cùng Bearer token và nội dung. Handler đọc token, tìm sessions còn hạn và người dùng tương ứng. Tiếp đó API kiểm tra người dùng có hàng trong

channel_members của kênh đích. Nếu không có, response 403 được trả trước khi nội dung được ghi. Nếu có quyền, body được chuẩn hóa, kiểm tra độ dài và replyTo, sau đó INSERT vào messages.

Sau khi ghi, server đọc lại hàng vừa tạo, bổ sung username và danh sách cảm xúc để tạo message DTO trả về. Client cập nhật danh sách tin và lần poll sau tiếp tục đồng bộ với server. Cấu trúc response thống nhất giúp cùng một thành phần hiển thị tin mới, tin đã sửa và tin tải từ lịch sử. Trường deleted_at được lọc khi đọc danh sách nên tin xóa mềm không xuất hiện trở lại.

Đối với tìm kiếm, chuỗi truy vấn được cắt về tối đa 100 ký tự. SQL kết hợp messages với channel_members theo userId của phiên, bỏ tin có deleted_at và so khớp nội dung bằng instr(lower(body), lower(q)). Kết quả được sắp theo id giảm dần, giới hạn 50 hàng. Điều kiện quyền nằm trong truy vấn, giúp tránh đưa tin của DM hoặc nhóm khác vào kết quả rồi mới lọc ở client.

5.6 Vận hành và truy cập từ thiết bị

Docker Compose mặc định công bố 127.0.0.1:8070 trên máy Windows. Cách bind này phù hợp khi chỉ dùng Android Emulator hoặc máy host. Android Emulator truy cập localhost của host qua địa chỉ 10.0.2.2, nên màn hình xác thực dùng URL http://10.0.2.2:8070. Trên iPhone thật, 10.0.2.2 không có ý nghĩa; người dùng nhập IP Wi-Fi của máy Windows và đặt HKCHAT_BIND=0.0.0.0 trước khi tạo lại container.

README hướng dẫn kiểm tra /api/health trước khi mở app. Response có status ok và product HKchat cho thấy API đã lắng nghe; nó không thay cho việc kiểm tra tài khoản và quyền. Docker volume giữ database sau khi dừng hoặc tạo lại container bình thường. Lệnh down -v xóa volume và vì vậy xóa dữ liệu đang lưu; báo cáo chỉ ghi thao tác này như một ranh giới vận hành, không xem đó là bước chạy thông thường.

Bản Expo Go nằm trong expo-preview và đồng bộ giao diện từ App.tsx trước khi chạy. Cách này cho phép kiểm thử giao diện và API trên iPhone từ Windows, trong khi project iOS native ở thư mục ios vẫn cần Xcode trên macOS để build và ký cài trực tiếp. Hai hình thức kiểm thử được phân biệt để không coi Expo Go là bản IPA native đã phát hành.

5.7 Danh mục endpoint của API

API trả JSON theo cùng cách cho client React Native và bộ test tích hợp. Health, register và login không cần phiên; các route còn lại phải có Bearer token. Endpoint theo kênh luôn kiểm tra membership trước khi đọc hoặc ghi. Endpoint theo tin tra channel_id của tin rồi kiểm tra thành viên kênh, sau đó mới xét tác giả nếu thao tác là sửa hoặc xóa.

Các endpoint được mô tả tiếp theo là giao diện thực tế trong server.mjs. Tham số :id đại diện UUID kênh hoặc id số của tin tùy đường dẫn. Status thành công phản ánh thao tác: 201 cho tài nguyên mới, 200 cho đọc hoặc cập nhật. Error handler dùng 400 cho đầu vào sai, 401 cho chưa đăng nhập, 403 cho không có quyền, 404 cho tài nguyên không tìm thấy và 409 cho trùng dữ liệu.

- **GET /api/health.** Quyền truy cập: Công khai. Mục đích: Kiểm tra server hoạt động.
- **POST /api/register.** Quyền truy cập: Công khai. Mục đích: Tạo tài khoản và phiên.
- **POST /api/login.** Quyền truy cập: Công khai. Mục đích: Xác thực và cấp phiên.
- **GET /api/me.** Quyền truy cập: Có token. Mục đích: Khôi phục người dùng.
- **POST /api/logout.** Quyền truy cập: Có token. Mục đích: Hủy phiên hiện tại.
- **GET /api/users.** Quyền truy cập: Có token. Mục đích: Danh sách người dùng khác.
- **GET /api/channels.** Quyền truy cập: Có token. Mục đích: Kênh đã tham gia.
- **GET /api/channels/public.** Quyền truy cập: Có token. Mục đích: Kênh công khai chưa tham gia.
- **POST /api/channels.** Quyền truy cập: Có token. Mục đích: Tạo kênh công khai.
- **POST /api/channels/:id/join.** Quyền truy cập: Có token. Mục đích: Tham gia kênh công khai.
- **POST /api/dm.** Quyền truy cập: Có token. Mục đích: Tạo hoặc mở DM.

- **POST /api/groups.** Quyền truy cập: Có token. Mục đích: Tạo nhóm qua API.
- **GET/POST /api/channels/:id/messages.** Quyền truy cập: Thành viên. Mục đích: Đọc hoặc gửi tin.
- **GET /api/messages/:id/thread.** Quyền truy cập: Thành viên. Mục đích: Đọc chuỗi trả lời.
- **PATCH/DELETE /api/messages/:id.** Quyền truy cập: Tác giả. Mục đích: Sửa hoặc xóa tin.
- **POST/DELETE /api/messages/:id/reactions.** Quyền truy cập: Thành viên. Mục đích: Thêm hoặc bỏ cảm xúc.
- **GET /api/search.** Quyền truy cập: Có token. Mục đích: Tìm trong kênh có quyền.

5.8 Cấu trúc request và response

Request đăng ký gửi JSON với username, email và password. Server không trả password_hash trong response; chỉ trả user và token dùng cho phiên. Request đăng nhập gửi identifier và password, trong đó identifier có thể là tên đăng nhập hoặc email. Client lưu token bằng Keychain thay vì đưa token vào URL. Các request đã đăng nhập gắn Authorization: Bearer <token> trong header.

Request gửi tin gửi body và, nếu đang trả lời, replyTo là số id của tin gốc. Response thành công chứa message DTO: id, channelId, userId, username, body, replyTo, createdAt, editedAt và reactions. Trường reactions là danh sách các mục emoji, count và me; me cho biết người đang xem đã thả cảm xúc đó. Cùng DTO được dùng khi đọc lịch sử và sau khi sửa hoặc thay đổi reaction.

Lỗi được đóng gói dưới khóa error để client hiển thị trong form hoặc banner. Error handler không trả stack trace cho lỗi nội bộ; thay vào đó trả 'Internal server error'. Điều này giữ response nhất quán và tránh đưa câu lệnh SQL hoặc đường dẫn tệp ra phía ứng dụng. Khi một token sai hoặc hết hạn, API trả 401 trước khi xử lý route nghiệp vụ.

- **Đăng ký.** *JSON đầu vào ví dụ:* {"username":"Lan","email":"lan@example.test","password":"********"}. *Phản hồi cốt lõi:* 201; user và token.
- **Đăng nhập.** *JSON đầu vào ví dụ:* {"identifier":"Lan","password":"********"}. *Phản hồi cốt lõi:* 200; user và token.
- **Tạo kênh.** *JSON đầu vào ví dụ:* {"name":"du-an-noi-bo"}. *Phản hồi cốt lõi:* 201; channel.
- **Mở DM.** *JSON đầu vào ví dụ:* {"userId":"<UUID người nhận>"}. *Phản hồi cốt lõi:* 200; channel.
- **Gửi reply.** *JSON đầu vào ví dụ:* {"body":"Đã nhận","replyTo":42}. *Phản hồi cốt lõi:* 201; message.
- **Sửa tin.** *JSON đầu vào ví dụ:* {"body":"Nội dung cập nhật"}. *Phản hồi cốt lõi:* 200; message.
- **Thả cảm xúc.** *JSON đầu vào ví dụ:* {"emoji":" 👍 "}. *Phản hồi cốt lõi:* 200; message với count mới.

5.9 Ma trận quyền truy cập

Token hợp lệ xác định người dùng nhưng không tự cho phép đọc mọi dữ liệu. Membership là điều kiện thứ hai cho kênh, tin, thread và reaction. Quyền tác giả là điều kiện thứ ba với PATCH/DELETE tin. Tách ba điều kiện này ngăn trường hợp người đã đăng nhập đoán id của một tin hoặc kênh khác rồi truy cập trực tiếp bằng API.

Với kênh công khai chưa tham gia, người dùng có thể thấy tên trong danh sách public nhưng chưa đọc tin. Họ phải gọi join để tạo membership. Với DM, endpoint join không chấp nhận và danh sách public không trả DM. Với nhóm, API public và join cũng không đưa nhóm riêng vào luồng tham gia mở. Những quy tắc này đã được test với tài khoản outsider.

- **Xem danh sách public.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Có. *Thành viên:* Có. *Tác giả tin:* Có.
- **Đọc tin kênh.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Có. *Tác giả tin:* Có.
- **Gửi tin.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Có. *Tác giả tin:* Có.

- **Xem thread.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Có. *Tác giả tin:* Có.
- **Thả cảm xúc.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Có. *Tác giả tin:* Có.
- **Sửa/xóa một tin.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Không. *Tác giả tin:* Có.
- **Tìm thấy tin trong kênh.** *Người chưa đăng nhập:* Không. *Ngoài kênh:* Không. *Thành viên:* Có. *Tác giả tin:* Có.

5.10 Chu trình migration và khởi động server

Khi process Node.js bắt đầu, server xác định đường dẫn database từ HKCHAT_DB hoặc dùng tệp mặc định cạnh server.mjs. Nó tạo thư mục cha nếu cần, mở DatabaseSync, bật foreign_keys và journal_mode WAL, rồi bảo đảm schema_migrations tồn tại. Danh sách tệp trong migrations được lọc theo mẫu tên số_*.sql và sắp xếp để áp dụng 001, 002, 003 theo thứ tự.

Với mỗi tệp, server đọc số phiên bản từ tên. Nếu schema_migrations đã có số đó, nó bỏ qua; nếu chưa, nó mở transaction, chạy SQL và ghi applied_at. Lỗi trong SQL làm rollback và dừng khởi động. Cách xử lý này tránh một bản migration được xem là hoàn tất khi chỉ một phần câu lệnh đã chạy. Đây là cơ chế đang có trong mã, không phải công cụ migration bên ngoài.

Sau migration, HTTP server lắng nghe trên 0.0.0.0:8070 bên trong container. Docker quyết định địa chỉ trên máy host qua HKCHAT_BIND. Healthcheck nội bộ gọi GET /api/health; Docker đánh dấu service healthy khi response thành công. Lần chạy gần nhất, API đã trả JSON status ok và product HKchat ở localhost:8070 sau khi container khởi động.

5.11 Kịch bản cài đặt lại trên Windows

Một máy mới cần Node.js đúng phiên bản, Docker Desktop, Android Studio, Android SDK và JDK theo README. Sau npm ci, người dùng mở Docker Desktop, đặt JAVA_HOME và ANDROID_HOME, rồi chạy docker compose -f hkchat/compose.yaml up -d. Lệnh curl tới /api/health xác nhận backend trước khi chạy npm run android. Cách kiểm tra theo thứ tự này tách lỗi dependency, Docker và client để dễ tìm nguyên nhân.

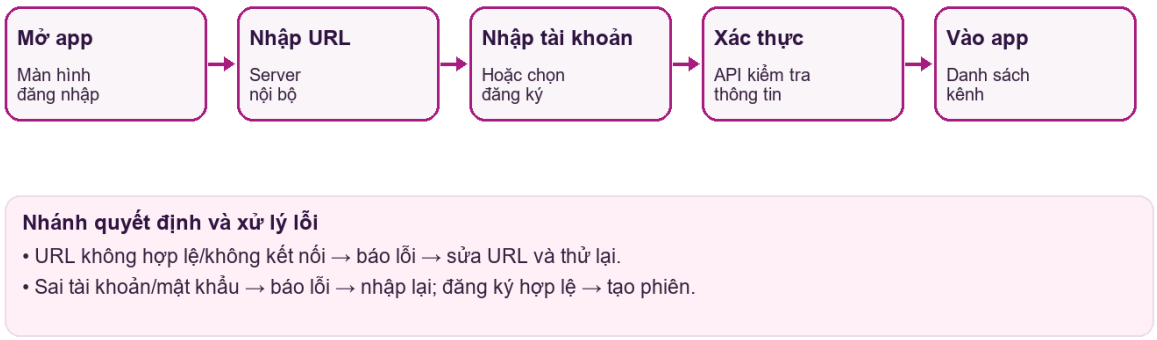
Nếu Docker CLI báo không tìm thấy pipe dockerDesktopLinuxEngine, engine chưa chạy; cần mở Docker Desktop và đợi trạng thái ready. Nếu Compose báo cổng 8070 đã được dùng, kiểm tra process hoặc thử lại khi Docker vừa khởi động. Trong lần kiểm tra ở máy thử, lệnh Compose thứ hai khởi động được container và healthcheck chuyển healthy, nên báo cáo không xem đó là lỗi mã nguồn HKchat.

Nếu health API trả ok nhưng app không đăng nhập được, kiểm tra URL trên thiết bị. Android Emulator cần 10.0.2.2; iPhone cần IP LAN của Windows và Docker bind ra mạng phù hợp. Việc thử Safari trên iPhone tới /api/health giúp phân biệt lỗi mạng với lỗi tài khoản. Đây là quy trình hỗ trợ kiểm thử thực tế đã ghi trong README, không đòi hỏi dịch vụ chat bên ngoài.

6. Luồng vận hành hệ thống

Hai sơ đồ mô tả đường đi trên giao diện: kết nối và xác thực, sau đó tìm tin và trả lời. Nhánh lỗi cho phép quay lại bước nhập dữ liệu tương ứng.

6.1 Kết nối server và đăng nhập



Hình 1. Kết nối server, xác thực và vào ứng dụng

6.2 Tìm tin và trả lời



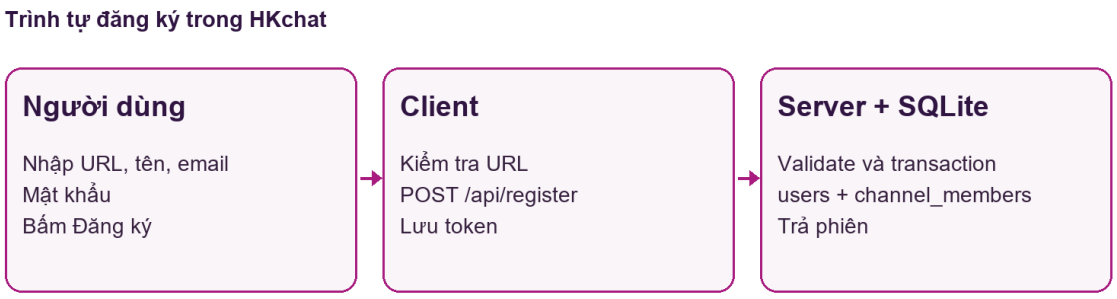
Hình 2. Tìm kiếm, mở hội thoại và trả lời tin

6.3 Luồng đăng ký và khởi tạo tài khoản

Người dùng mở HKchat và thấy màn hình nhập địa chỉ server, tên đăng nhập hoặc email, mật khẩu. Khi chọn Đăng ký, form có thêm email và tên đăng nhập. Client loại dấu / cuối địa chỉ và kiểm tra URL trước khi gọi API. Nếu địa chỉ sai, lỗi được hiển thị ngay trong form; nếu kết nối thất bại, người dùng có thể sửa IP hoặc cổng rồi thử lại.

Server chuẩn hóa username bằng trim và email bằng trim/lowercase, kiểm tra định dạng rồi tra users để phát hiện trùng. Khi dữ liệu hợp lệ, transaction tạo user và thêm người đó vào General và Random. Nếu một trong các thao tác thất bại, transaction rollback. Sau commit, server cấp token phiên và trả về user. Client lưu token bằng Keychain và chuyển sang danh sách kênh.

Một tài khoản mới vì vậy có hai kênh mặc định. Điều này là một quyết định trong mã nguồn, không phụ thuộc dữ liệu mẫu. Nếu username/email đã tồn tại, API trả 409; nếu định dạng không đạt, trả 400. Hai nhánh lỗi đều giữ người dùng ở form để sửa đầu vào, không tạo phiên dở dang.



Hình. Trình tự từ form đăng ký tới database

- **1.** *Tác nhân:* Người dùng. *Điều kiện và kết quả:* Nhập URL server, tên, email và mật khẩu.
- **2.** *Tác nhân:* Client. *Điều kiện và kết quả:* Kiểm tra URL; gửi POST /api/register.
- **3.** *Tác nhân:* Server. *Điều kiện và kết quả:* Kiểm tra độ dài tên, email, mật khẩu và trùng lặp.
- **4.** *Tác nhân:* SQLite. *Điều kiện và kết quả:* Ghi users và membership General/Random trong transaction.
- **5.** *Tác nhân:* Client. *Điều kiện và kết quả:* Nhận token, lưu phiên, tải danh sách kênh.

6.4 Luồng đăng nhập, khôi phục phiên và đăng xuất

Khi đăng nhập, client gửi identifier và password tới /api/login. Server tìm tài khoản theo username hoặc email rồi đối chiếu mật khẩu đã băm. Sai thông tin được phản hồi bằng HTTP 401. Đúng thông tin thì server tạo token mới, gửi về user và thời điểm tạo; client lưu token với service hkchat.local trong Keychain, đồng thời lưu URL server dưới service hkchat.server.

Khi app mở lại, effect khởi tạo đọc URL đã lưu trước rồi đọc token. Client gọi GET /api/me bằng token đó; nếu phiên hợp lệ, user và danh sách kênh được khôi phục, người dùng không phải nhập lại mật khẩu. Nếu token hết hạn hoặc API lỗi, client xóa token đã lưu và trả về màn xác thực. Cơ chế này dựa vào việc server là nguồn xác nhận phiên, không chỉ dựa vào thông tin trên điện thoại.

Khi đăng xuất, client gọi POST /api/logout, server xóa token_hash của phiên hiện tại. Client xóa thông tin phiên và trở về màn xác thực. Địa chỉ server được lưu riêng để lần vào sau không phải nhập lại IP nếu môi trường mạng không thay đổi. Trong Figma, hộp thoại xác nhận đăng xuất là một trạng thái riêng trước khi trở về màn đăng nhập.

6.5 Luồng tạo kênh và tham gia kênh

Trang Kênh phân biệt các kênh người dùng đã tham gia và kênh công khai khác. Khi tạo, client gửi tên kênh mới. Server kiểm tra độ dài 2–80 ký tự, tạo UUID, ghi kênh với kind public và thêm người tạo vào channel_members trong cùng transaction. Sau khi API thành công, client làm mới danh sách và mở kênh để người dùng có thể bắt đầu gửi tin.

Khi tham gia, client gửi POST tới /api/channels/:id/join. Server tìm đúng một kênh public không thuộc group_chats; kênh không hợp lệ trả 404. Lệnh INSERT OR IGNORE trong membership giúp bấm lại không tạo bản ghi trùng. Danh sách kênh có thể tham gia sau đó được làm mới, vì kênh vừa tham gia chuyển sang nhóm Kênh của tôi.

Với hội thoại nhóm, server có API /api/groups để tạo kênh cùng danh sách thành viên và bản ghi group_chats. Tuy nhiên giao diện hiện tại chưa có biểu mẫu tự tạo nhóm; bản chụp dùng nhóm bốn người được chuẩn bị trong database mẫu. Việc phân biệt API và giao diện giúp báo cáo phản ánh đúng chức năng mà người dùng thao tác được.

6.6 Luồng DM và kiểm soát người tham gia

Tab Mọi người tải danh sách user khác tài khoản hiện tại, sắp theo tên. Chạm vào một người gửi userId lên /api/dm. Server kiểm tra người nhận tồn tại, không phải chính người gửi; sau đó ghép hai id theo thứ tự sắp xếp thành dm_key. Nếu đã có DM cho cặp này, API trả lại kênh cũ; nếu chưa, transaction tạo channel kind dm và hai hàng channel_members.

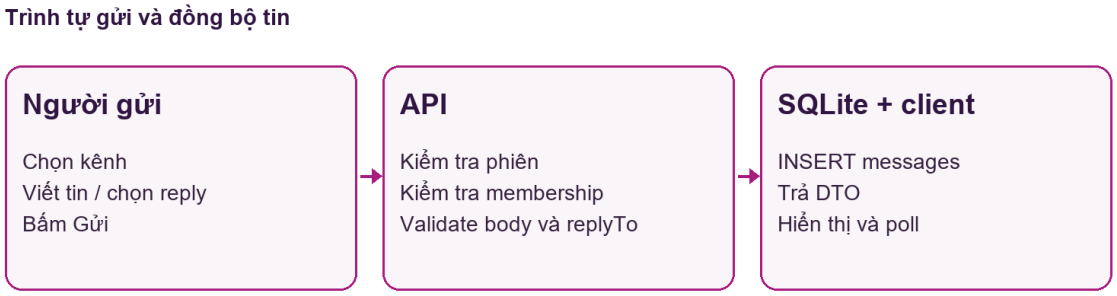
Sau khi mở DM, màn hội thoại dùng cùng thành phần tin nhắn như kênh công khai. Quyền đọc được xác định theo channel_members. Một người thứ ba có token hợp lệ nhưng không thuộc DM vẫn nhận HTTP 403 khi gọi API đọc tin. Tìm kiếm cũng JOIN qua channel_members, do đó nội dung DM không xuất hiện trong kết quả của người thứ ba.

6.7 Luồng gửi tin, trả lời và thread

Khi người dùng viết tin và bấm Gửi, client gửi body tới endpoint messages của kênh đang mở. Nếu đang trả lời, request có thêm replyTo là id của tin được chọn. Server kiểm tra membership và tính hợp lệ của tin đích trong cùng kênh. Sau khi ghi, response trả bản tin đầy đủ để client có thể hiển thị ngay, trong khi cơ chế poll định kỳ đồng bộ các tin mới từ người khác.

Trên giao diện, nhấn giữ một tin mở menu với Trả lời và các hành động tương ứng. Chọn Trả lời đưa nội dung trích dẫn lên thanh ngữ cảnh phía trên ô soạn; người dùng có thể hủy để trở về chế độ gửi tin thường. Trong danh sách, một tin có replyTo hiển thị đoạn trích của tin gốc nếu tin đó còn ở dữ liệu đã tải. Màn hình thread dùng API đệ quy tìm gốc và các tin con khi cần xem chuỗi liên quan.

API thread dùng hai CTE đệ quy: ancestors đi ngược reply_to đến tin gốc, descendants đi từ gốc xuống các phản hồi. Kết quả bỏ tin đã xóa và giới hạn 100 tin. Việc kiểm tra membership diễn ra trước truy vấn, nên biết id tin của kênh khác không đủ để đọc nội dung thread.



Hình. Luồng gửi tin và phản hồi từ API

6.8 Luồng sửa, xóa, cảm xúc và tìm kiếm

Chọn Sửa trên tin của mình chuyển ô soạn sang chế độ chỉnh sửa, giữ bản gốc ở thanh ngữ cảnh. Bấm Lưu gửi PATCH với nội dung mới. Server kiểm tra tác giả và độ dài trước khi cập nhật body, edited_at. Chọn Xóa hiển thị xác nhận; nếu người dùng chấp nhận, client gọi DELETE và server ghi deleted_at. Tin biến mất khỏi danh sách đọc mà không xóa vật lý hàng dữ liệu.

Bộ chọn cảm xúc nằm dưới tin. Chạm một emoji gửi POST; chạm lại cảm xúc của chính mình gửi DELETE theo emoji. Server trả lại DTO với count và me mới, nên giao diện có thể đổi trạng thái được chọn. Người dùng không thuộc kênh không thể thả cảm xúc dù biết id tin.

Ở tab Tìm kiếm, người dùng nhập từ khóa và bấm Tìm kiếm. Client gửi q đã mã hóa URL; API trả tối đa 50 tin phù hợp trong các kênh thành viên. Chạm một kết quả lấy channelId từ response, tìm kênh tương

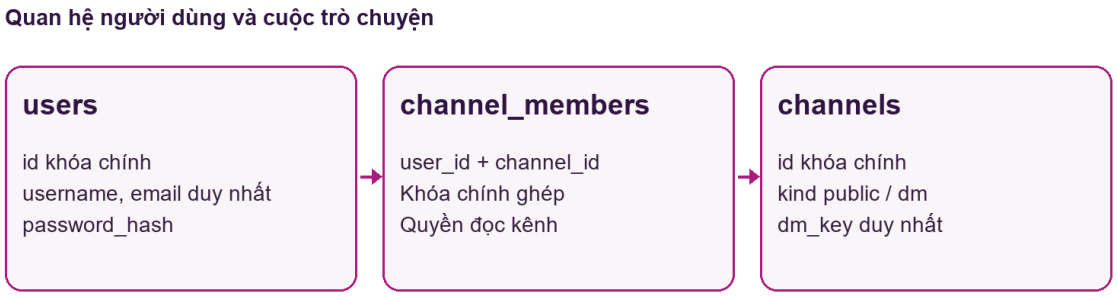
ứng trong danh sách đã tải rồi mở hội thoại. Đây là luồng đang có trong app; bản hiện tại mở hội thoại chứ chưa tự cuộn đến đúng vị trí tin tìm thấy.

7. Mô hình và lược đồ dữ liệu

7.1 Quan hệ dữ liệu và khóa ngoại

Tập dữ liệu cốt lõi gồm users, sessions, channels, channel_members, messages, reactions và group_chats. users là gốc của phiên và quyền tác giả. channels là gốc của hội thoại; channel_members nối người dùng với hội thoại. messages thuộc một kênh và một tác giả, còn reply_to tự tham chiếu tới một tin khác. reactions nối người dùng với tin, cho phép tổng hợp số lượt theo emoji.

SQLite bật PRAGMA foreign_keys = ON để các tham chiếu được kiểm tra. Một số quan hệ có ON DELETE CASCADE, ví dụ xóa kênh sẽ kéo theo membership và messages; báo cáo không mô tả xóa kênh như một thao tác UI vì client hiện chưa cung cấp thao tác đó. Các khóa chính ghép ngăn thành viên bị thêm hai lần vào cùng kênh và ngăn một người gắn cùng emoji hai lần vào một tin.



Hình. Quan hệ users, channel_members và channels

7.2 Từ điển dữ liệu bảng users và sessions

Bảng users lưu định danh UUID, tên đăng nhập, email, hash mật khẩu và thời điểm tạo. COLLATE NOCASE cùng UNIQUE được đặt cho username và email, giúp kiểm tra trùng không phụ thuộc chữ hoa/chữ thường. Server vẫn kiểm tra dữ liệu đầu vào trước khi INSERT để trả thông báo nghiệp vụ dễ hiểu hơn lỗi ràng buộc thô của SQLite.

Bảng sessions không lưu token thuần. Client nhận token 32 byte ngẫu nhiên ở dạng hex; server băm SHA-256 và lưu token_hash làm khóa chính, cùng user_id và expires_at. Mỗi request có Authorization: Bearer sẽ băm token nhận được rồi tìm phiên chưa hết hạn. Đăng xuất xóa hàng ứng với token hiện tại.

Bảng.trường	Kiểu và ràng buộc	Mục đích
users.id	TEXT, PRIMARY KEY	UUID của tài khoản
users.username	TEXT, NOT NULL, UNIQUE, NOCASE	Tên hiển thị và định danh đăng nhập
users.email	TEXT, NOT NULL, UNIQUE, NOCASE	Email đăng nhập
users.password_hash	TEXT, NOT NULL	Chuỗi salt:hash scrypt
users.created_at	INTEGER, NOT NULL	Mốc tạo tài khoản tính bằng ms
sessions.token_hash	TEXT, PRIMARY KEY	Hash SHA-256 của token phiên
sessions.user_id	TEXT, FOREIGN KEY	Người sở hữu phiên

sessions.expires_at	INTEGER, NOT NULL	Thời điểm hết hạn phiên

7.3 Từ điển dữ liệu bảng channels và channel_members

channels chứa cả kênh công khai và DM. kind chỉ nhận public hoặc dm theo CHECK trong schema. DM có dm_key duy nhất là hai userId đã được sắp xếp và ghép; với kênh công khai, dm_key để trống. group_chats đánh dấu một channel là nhóm, còn DTO của API trả kind group cho client để hiển thị đúng. Cách mô hình hóa này tận dụng chung messages và membership cho nhiều loại hội thoại.

channel_members dùng khóa chính (channel_id, user_id). Khi đăng ký, hai hàng được thêm cho General và Random. Khi tạo kênh, người tạo được thêm ngay. Khi tham gia kênh công khai hoặc tạo DM/nhóm, bảng này cập nhật quyền đọc tin. Truy vấn danh sách kênh chỉ lấy channel có membership của người đăng nhập.

Bảng.trường	Kiểu và ràng buộc	Mục đích
channels.id	TEXT, PRIMARY KEY	UUID cuộc trò chuyện
channels.name	TEXT, NOT NULL	Tên kênh hoặc DM
channels.kind	TEXT, CHECK public/dm	Loại lưu ở schema
channels.dm_key	TEXT, UNIQUE	Cặp người dùng DM
channels.created_at	INTEGER, NOT NULL	Mốc tạo kênh
channel_members.channel_id	TEXT, PK ghép, FK	Cuộc trò chuyện
channel_members.user_id	TEXT, PK ghép, FK	Thành viên
channel_members.joined_at	INTEGER, NOT NULL	Mốc tham gia
group_chats.channel_id	TEXT, PRIMARY KEY, FK	Kênh được đánh dấu là nhóm
group_chats.creator_id	TEXT, FOREIGN KEY	Người tạo nhóm

7.4 Từ điển dữ liệu bảng messages và reactions

messages dùng id INTEGER PRIMARY KEY AUTOINCREMENT để tăng theo thứ tự ghi. channel_id gắn tin với hội thoại; user_id gắn tác giả. body có ràng buộc CHECK độ dài 1–4000 ký tự. reply_to là khóa ngoại tự tham chiếu; nó cho phép một tin trả lời tin khác và hỗ trợ truy vấn thread bằng CTE đệ quy. created_at, edited_at và deleted_at phân biệt tin gốc, tin sửa và tin đã xóa mềm.

reactions có khóa chính ghép (message_id, user_id, emoji). Khi server tạo DTO, truy vấn GROUP BY message_id và emoji để tính count; biểu thức CASE đánh dấu me nếu người đang xem đã chọn cảm xúc đó. Nhờ vậy client không phải tự tổng hợp từng người thả emoji hoặc tải toàn bộ bản ghi reaction.

Bảng.trường	Kiểu và ràng buộc	Mục đích
messages.id	INTEGER, AUTOINCREMENT PK	Thứ tự và định danh tin
messages.channel_id	TEXT, NOT NULL, FK	Hội thoại chứa tin
messages.user_id	TEXT, NOT NULL, FK	Tác giả

messages.body	TEXT, CHECK 1–4000 ký tự	Nội dung
messages.reply_to	INTEGER, FK tự tham chiếu	Tin được trả lời
messages.created_at	INTEGER, NOT NULL	Mốc tạo
messages.edited_at	INTEGER, NULL	Mốc sửa
messages.deleted_at	INTEGER, NULL	Mốc xóa mềm
reactions.message_id	INTEGER, PK ghép, FK	Tin được thả cảm xúc
reactions.user_id	TEXT, PK ghép, FK	Người thả cảm xúc
reactions.emoji	TEXT, PK ghép	Biểu tượng cảm xúc

7.5 Chỉ mục và ràng buộc toàn vẹn

Chỉ mục sessions_user_idx phục vụ các thao tác liên quan tới phiên của một người. channel_members_user_idx hỗ trợ truy vấn danh sách kênh của người dùng. messages_channel_idx và messages_user_idx giúp truy vấn tin theo kênh hoặc tác giả; reactions và group_chats đã có khóa chính phù hợp với các phép tra cứu thường dùng. Đây là chỉ mục có trong migration, không phải kết quả benchmark hiệu năng.

Giao dịch BEGIN/COMMIT bao quanh việc tạo người dùng và kênh mặc định, tạo kênh với membership, tạo nhóm với toàn bộ thành viên và tạo DM. Nếu một INSERT lỗi, ROLLBACK tránh để lại kênh không có thành viên hoặc tài khoản chỉ được tạo một phần. Error handler đổi lỗi SQLITE_CONSTRAINT thành HTTP 409 và lỗi nội bộ thành thông báo chung để response không làm lộ chi tiết truy vấn.

Migration được theo dõi trong schema_migrations(version, applied_at). Khi khởi động, server đọc các file có tên dạng số_thứ-tự.sql, bỏ qua phiên bản đã ghi nhận và áp dụng file mới trong transaction. Cơ chế này giữ database hiện có khi nâng cấp; thay đổi schema mới nên thêm migration thay vì sửa file migration đã được chạy.

8. Thiết kế giao diện và trải nghiệm tương tác

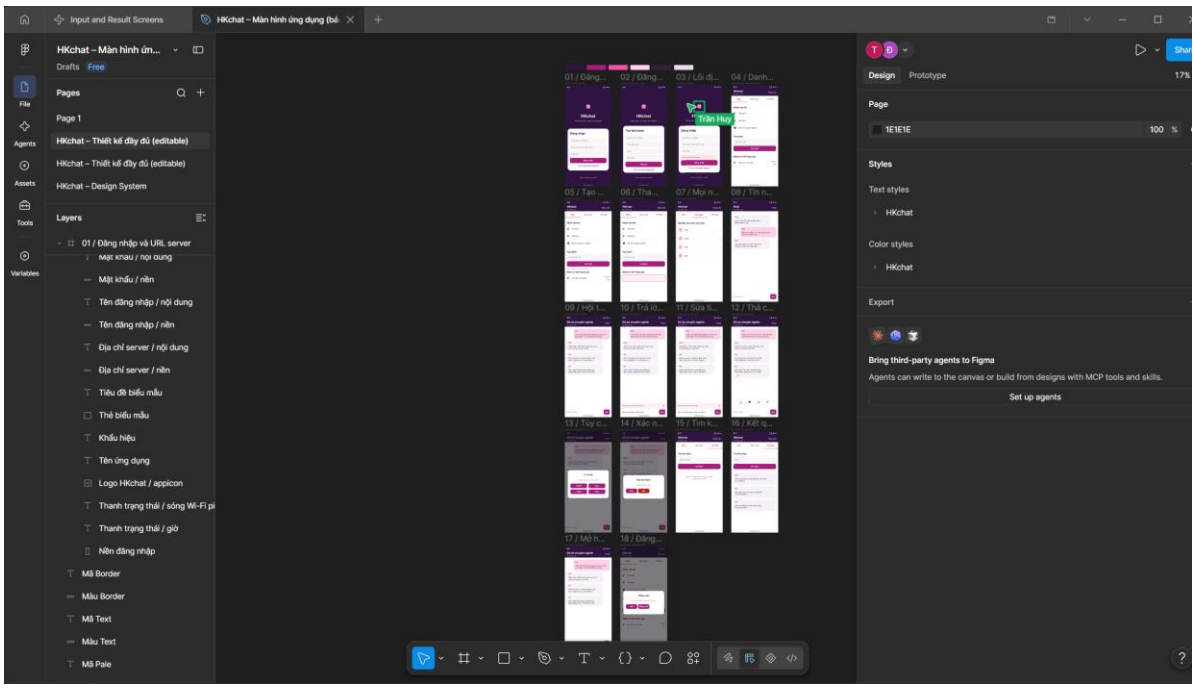
8.1 Hệ thống thiết kế

Thiết kế dùng tím đậm ở màn hình xác thực và thanh tiêu đề, tím hồng cho nút chính và trạng thái chọn, nền sáng cho nội dung hội thoại. Figma có Color Styles, Text Styles và component variants cho nút, ô nhập, tab và bong bóng tin. Logo dự án lấy từ appicon.webp.

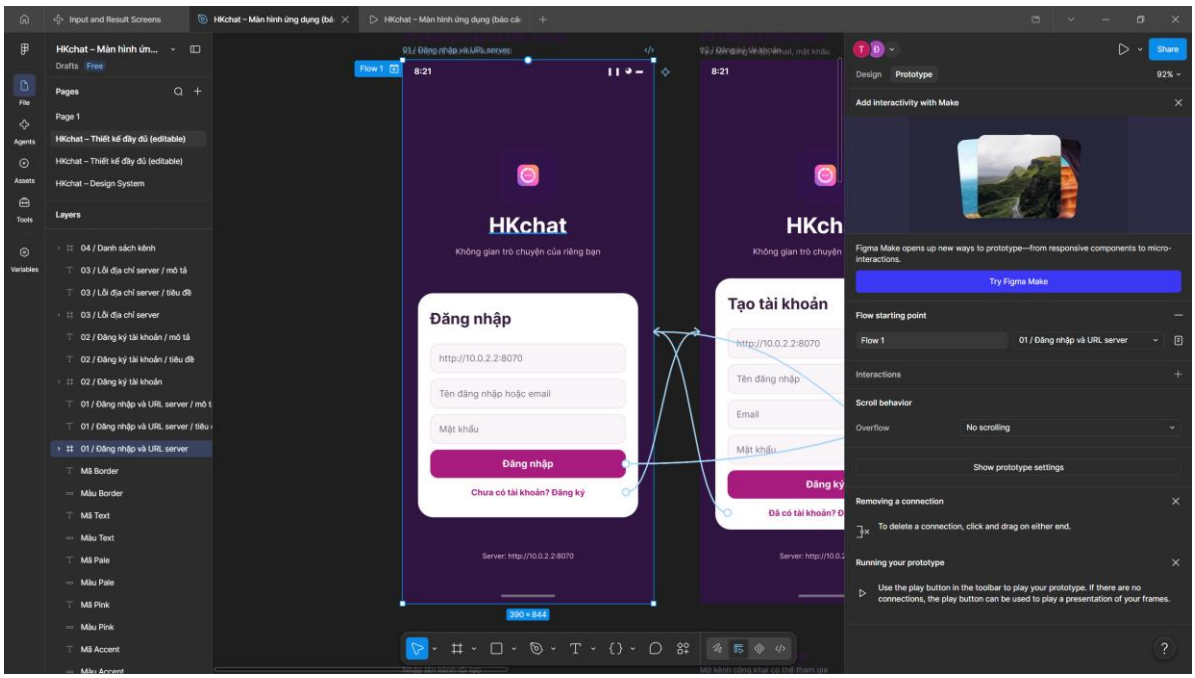
- **Nền đậm.** *Màu:* #301542. *Ứng dụng:* Màn hình xác thực, thanh tiêu đề.
- **Màu nhấn.** *Màu:* #A81C7F. *Ứng dụng:* Nút chính, tên tác giả, tab được chọn.
- **Hồng.** *Màu:* #F553AB. *Ứng dụng:* Điểm nhấn tương tác.
- **Hồng nhạt.** *Màu:* #FFD1E9. *Ứng dụng:* Bong bóng tin của người dùng.
- **Chữ.** *Màu:* #37213F. *Ứng dụng:* Nội dung trên nền sáng.

Figma dùng font Inter Regular/Bold, khung 390 × 844 px và lưới khoảng cách 8 px. Bộ thiết kế gồm 18 trạng thái màn hình chỉnh sửa được theo layer. Sau khi chạy plugin, nhóm xác nhận có thể đi tới 18/18 màn hình từ Flow 1.

Liên kết Figma: Mở thiết kế và prototype HKchat



Hình 3. Trang thiết kế và các style trong Figma



Hình 4. Flow 1 và các đường nối trong prototype

8.2 Cấu trúc điều hướng

Sau đăng nhập, người dùng chọn một trong ba tab Kênh, Mọi người và Tìm kiếm. Kênh hiển thị cuộc trò chuyện và thao tác tạo/tham gia. Mọi người mở DM. Tìm kiếm trả danh sách tin và mở lại hội thoại. Các thao tác reply, sửa/xóa và cảm xúc được đặt trong màn hình tin nhắn.

8.3 Nguyên tắc bố cục và phản hồi

Bộ màn hình lấy vùng tiêu đề tím làm mốc nhận diện và vùng nội dung sáng làm nơi đọc tin. Màn xác thực dùng nền tím toàn trang, thẻ biểu mẫu trắng và nút tím hồng nổi bật để chỉ rõ hành động chính. Sau đăng nhập, thanh tab ba mục nằm ngay dưới tiêu đề; tab được chọn có màu nhấn và chỉ báo dưới chân. Cấu trúc lặp lại giữa các trang giúp người dùng không phải học lại vị trí điều hướng.

Trong hội thoại, bong bóng của người đang dùng có nền hồng nhạt, tin của người khác có nền sáng khác. Tên tác giả được đặt ở đầu bong bóng. Khi trả lời, trích dẫn nằm trước nội dung mới; khi sửa, thanh ngữ cảnh cho biết đang sửa tin nào. Nút cảm xúc đặt sát tin để người dùng thấy hành động gắn với nội dung cụ thể, không phải toàn kênh.

Ứng dụng dùng TextInput với nhãn accessibility cho địa chỉ server, tên, email, mật khẩu, nội dung tin và từ khóa tìm kiếm. Các nút chính dùng accessibilityRole button. Thông báo lỗi có chữ hiển thị trong form hoặc banner, không chỉ đổi màu. Đây là các quyết định triển khai có thể kiểm tra trong App.tsx; báo cáo không gán điểm accessibility định lượng khi chưa chạy công cụ đo chuyên dụng.

Figma dùng khung 390 × 844 px và ghi quy tắc khoảng cách theo bội số 8 px. Các thành phần được tách layer để sửa nội dung, màu và hình khối. Trang Design System bổ sung Button, Input, Tab, Message và Avatar cùng các biến thể trạng thái. File thiết kế đóng vai trò mô phỏng hướng đi; ảnh Android trong báo cáo chứng minh một số màn hình của ứng dụng đang chạy.

- **Button.** *Trạng thái trong Figma:* Primary, Secondary, Disabled, Destructive. *Cách dùng:* Hành động gửi, xác nhận, hủy hoặc xóa.
- **Input.** *Trạng thái trong Figma:* Default, Focused, Error. *Cách dùng:* Đăng nhập, tạo kênh, tìm kiếm.
- **Tab.** *Trạng thái trong Figma:* Selected, Default. *Cách dùng:* Kênh, Mọi người, Tìm kiếm.
- **Message.** *Trạng thái trong Figma:* Mine, Other. *Cách dùng:* Phân biệt người gửi trong hội thoại.

- **Avatar.** *Trạng thái trong Figma:* Member. *Cách dùng:* Danh sách người dùng.

8.4 Các trạng thái xác thực và kênh

Màn 01 là điểm bắt đầu Flow 1: form nhập URL server, tên hoặc email và mật khẩu. Màn 02 thêm trường tên đăng nhập và email để tạo tài khoản. Màn 03 minh họa lỗi kết nối hoặc URL không dùng được. Ba trạng thái chung khung nhận diện và thẻ form nên người dùng hiểu đây là một bước xác thực với các trường hợp khác nhau, không phải ba ứng dụng độc lập.

Màn 04 liệt kê kênh đã tham gia và ba tab chính. Màn 05 minh họa nhập tên kênh mới, bấm Tạo kênh và chuyển sang hội thoại. Màn 06 cho thấy kênh công khai chưa tham gia với hành động Tham gia. Khi đánh giá prototype, luồng từ màn danh sách tới các trạng thái tạo, tham gia và trở lại kênh đã được nối bằng tương tác của plugin Figma.

Trong ứng dụng thật, tài khoản mới có General và Random nhờ server tạo membership trong transaction đăng ký. Kênh có thể tham gia được tải từ endpoint riêng, chỉ trả kênh public mà người dùng chưa là thành viên và không phải group chat. Vì vậy phần hiển thị trên màn 04–06 có quy tắc dữ liệu đi kèm, không chỉ là bố cục minh họa.

- **01.** *Nội dung:* Đăng nhập và URL server. *Hành động chính:* Đăng nhập; chuyển sang Đăng ký.
- **02.** *Nội dung:* Đăng ký. *Hành động chính:* Tạo tài khoản; quay lại Đăng nhập.
- **03.** *Nội dung:* Lỗi địa chỉ server. *Hành động chính:* Sửa URL hoặc thông tin và thử lại.
- **04.** *Nội dung:* Danh sách kênh. *Hành động chính:* Mở kênh, đổi tab, tạo/tham gia.
- **05.** *Nội dung:* Tạo kênh mới. *Hành động chính:* Nhập tên rồi tạo.
- **06.** *Nội dung:* Tham gia kênh. *Hành động chính:* Bấm Tham gia để mở hội thoại.

8.5 Mọi người, DM và hội thoại nhóm

Màn 07 là danh sách thành viên để bắt đầu DM. Dòng người dùng có avatar chữ đầu và ký hiệu mở ở bên phải. Màn 08 là hội thoại riêng giữa Huy và Khải, dùng cùng định dạng tin với màn nhóm. Tên cuộc trò chuyện thay cho tên ứng dụng ở thanh tiêu đề; nút Trở lại đưa người dùng về danh sách.

Màn 09 trình bày hội thoại nhóm bốn người. Dữ liệu mẫu dùng các tên Huy, Khải, Tân và Đạt với nội dung trò chuyện về đồ án để dễ nhận ra nhiều tác giả; đó là nội dung kiểm thử, không giới hạn HKchat cho chủ đề học tập. Mục đích sử dụng sản phẩm vẫn là nhắn tin nội bộ cho nhóm làm việc. Tin của Huy dùng nền hồng nhạt, các tin còn lại dùng nền sáng trung tính.

Khi mở một kênh hoặc DM, client đặt current, tải tối đa 100 tin ban đầu và lặp yêu cầu lấy tin mỗi hai giây. Đổi hội thoại sẽ hủy timer cũ, xóa bản nháp, trạng thái reply/sửa và bộ chọn emoji để tránh thao tác nhầm trên cuộc trò chuyện trước. Những chi tiết này liên quan trực tiếp tới tính nhất quán của màn 08–09.

8.6 Các thao tác trên tin

Màn 10 hiển thị thanh trả lời với trích dẫn ngắn; ô soạn chứa nội dung mới và nút Gửi. Màn 11 dùng cùng vị trí thanh ngữ cảnh nhưng nhãn chuyển thành Sửa và nút thành Lưu. Người dùng có thể bấm × để hủy thao tác. Việc chia hai trạng thái cho phép reviewer thấy sự khác biệt giữa tạo reply mới và cập nhật tin đã có.

Màn 12 hiển thị bộ emoji và số lượt phản ứng. Màn 13 là menu tùy chọn sau nhấn giữ tin, gồm Trả lời, Sửa, Thêm và Hủy trong prototype; thao tác app thật tùy quyền của người dùng đối với tin. Màn 14 là hộp xác nhận Xóa, tạo một bước kiểm tra trước khi gửi DELETE. Trong server, xóa chỉ ghi deleted_at và tác giả mới được phép thực hiện.

Các trạng thái 10–14 dùng cùng một nền hội thoại để người xem hiểu thao tác bắt đầu từ tin đã chọn. Figma nối những nút này tới trạng thái tiếp theo, còn Android screenshot cho thấy trực tiếp thanh reply trong ứng dụng đang chạy. Nhờ đó case study có cả quyết định thiết kế lẫn bằng chứng triển khai.

- **10.** *Trạng thái:* Trả lời. *Quy tắc nghiệp vụ liên quan:* replyTo trỏ tin còn tồn tại trong cùng kênh.
- **11.** *Trạng thái:* Sửa. *Quy tắc nghiệp vụ liên quan:* Chỉ tác giả PATCH được tin.
- **12.** *Trạng thái:* Thả cảm xúc. *Quy tắc nghiệp vụ liên quan:* Một emoji/người/tin là duy nhất.
- **13.** *Trạng thái:* Tùy chọn tin. *Quy tắc nghiệp vụ liên quan:* Hành động phụ thuộc tin và quyền.
- **14.** *Trạng thái:* Xác nhận xóa. *Quy tắc nghiệp vụ liên quan:* Chỉ tác giả DELETE được; server xóa mềm.

8.7 Tìm kiếm và kết thúc phiên

Màn 15 là form từ khóa với hướng dẫn khi chưa tìm. Màn 16 hiển thị kết quả gồm tên tác giả và đoạn nội dung, đủ để người dùng chọn đúng tin. Màn 17 là hội thoại sau khi mở kết quả; app thật mở kênh tương ứng. Vì client chưa tự cuộn đến id tin được chọn, báo cáo chỉ khẳng định hành vi mở hội thoại chứ không khẳng định định vị chính xác tin trong lịch sử.

Màn 18 là hộp thoại xác nhận Đăng xuất trên nền danh sách kênh. Chọn Hủy giữ nguyên phiên; chọn Đăng xuất trở về form xác thực. Đây là điểm kết thúc tự nhiên của Flow 1. Plugin Figma đã kiểm đếm và người dùng xác nhận có thể đi tới 18/18 màn hình từ điểm bắt đầu.

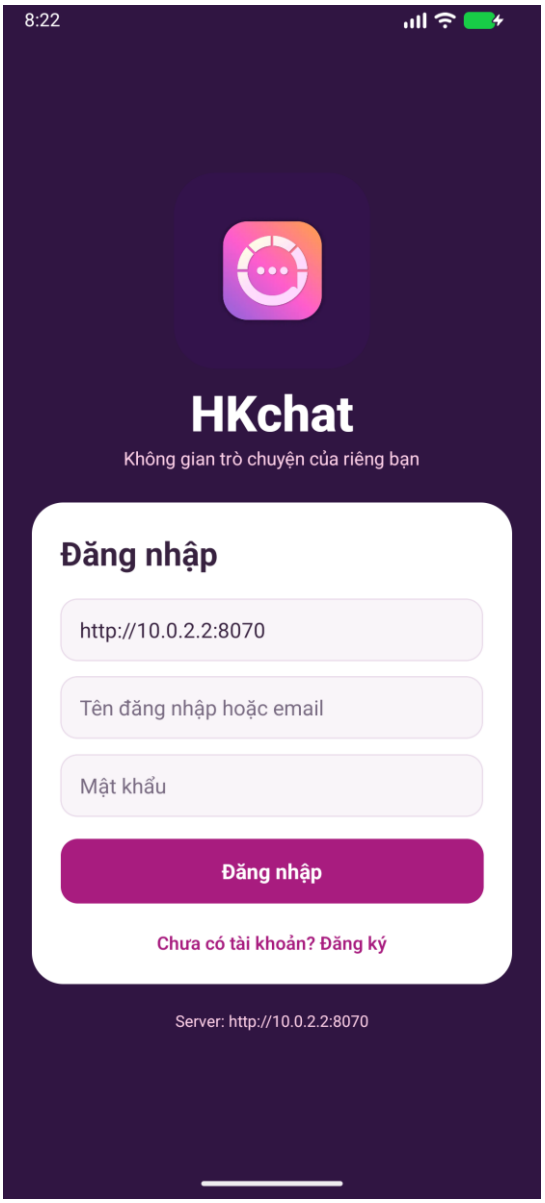
Bộ ảnh case study dùng các màn ứng dụng thật cho đăng nhập, kênh, hội thoại, reply, tìm kiếm và mở kết quả; ảnh Figma dùng để trình bày độ bao phủ thiết kế. Phân biệt hai loại ảnh này giúp người xem biết phần nào là app chạy thật và phần nào là prototype tương tác.

- **15.** *Nội dung:* Nhập từ khóa. *Liên hệ chức năng:* GET /api/search?q=...
- **16.** *Nội dung:* Danh sách kết quả. *Liên hệ chức năng:* Tối đa 50 tin trong kênh có quyền.
- **17.** *Nội dung:* Mở hội thoại. *Liên hệ chức năng:* Dùng channelId của kết quả.
- **18.** *Nội dung:* Xác nhận đăng xuất. *Liên hệ chức năng:* POST /api/logout và xóa phiên client.

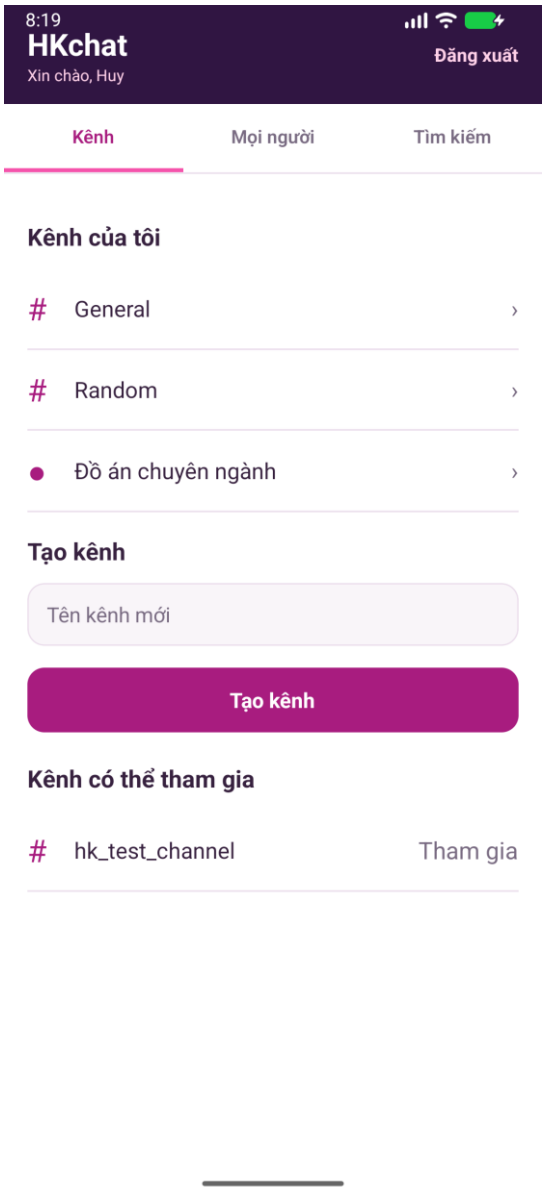
8.8 Các khung nhìn giao diện tiêu biểu

Các hình dưới đây chụp từ bản chạy trên Android Emulator với dữ liệu mẫu của nhóm.

Hình 5. Nhập URL server và đăng nhập



Hình 6. Danh sách kênh, nhóm

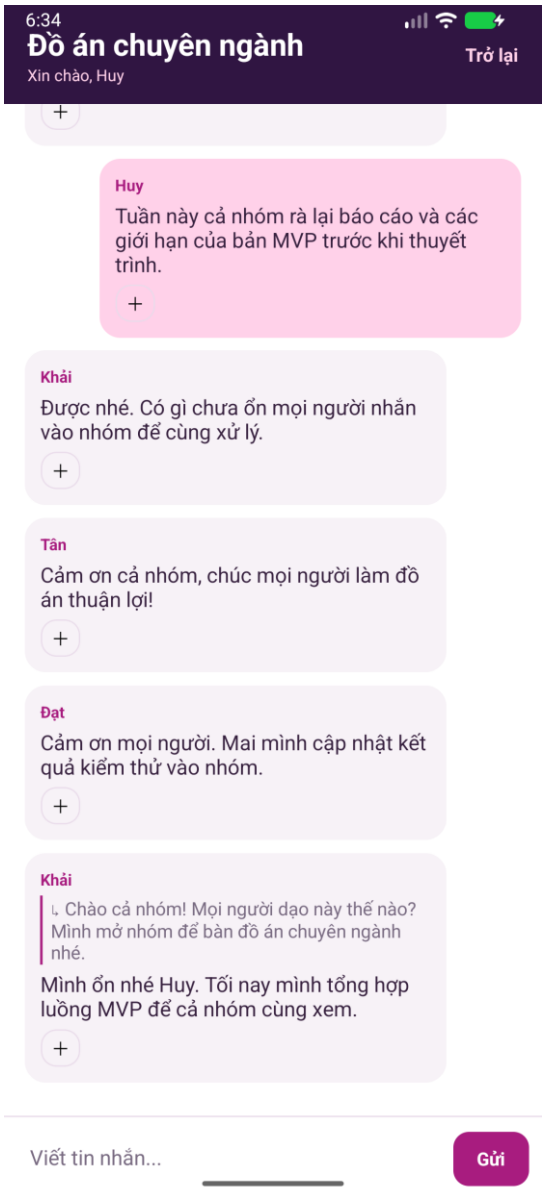


Khung nhìn 1 — xác thực: Android Emulator dùng URL http://10.0.2.2:8070. Người dùng nhập tên hoặc email và mật khẩu; client gọi API rồi lưu phiên khi thành công. Lỗi xác thực và kết nối hiển thị ngay trong form. Khung nhìn 2 — danh sách: sau đăng nhập, tab Kênh liệt kê kênh đã tham gia, nhóm mẫu, ô tạo kênh và các kênh công khai có thể tham gia. API trả hai danh sách theo quyền riêng biệt.

Hình 7. Hội thoại nhóm bốn người

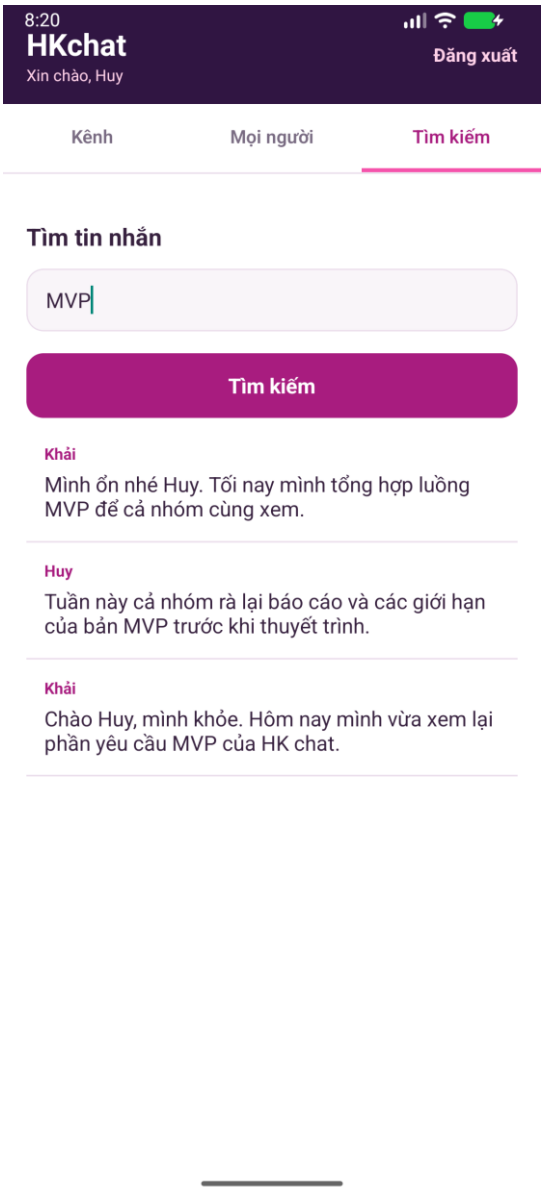


Hình 8. Trả lời tin có ngữ cảnh



Khung nhìn 3 — hội thoại: dữ liệu mẫu có Huy, Khải, Tân và Đạt; nội dung về đồ án chỉ dùng để minh họa nhiều tác giả trong nhóm làm việc. Tin của người đăng nhập có màu khác để dễ nhận diện. Khung nhìn 4 — trả lời: nhấn giữ một tin rồi chọn Trả lời làm xuất hiện thanh trích dẫn; request mang replyTo. Server kiểm tra tin gốc còn tồn tại và thuộc cùng kênh trước khi lưu.

Hình 9. Tìm tin theo từ khóa



Hình 10. Mở hội thoại từ kết quả



Khung nhìn 5 — tìm kiếm: nhập từ khóa và nhận tối đa 50 tin mới nhất trong phạm vi kênh được phép xem; tin xóa không xuất hiện. Khung nhìn 6 — mở kết quả: chọn kết quả để quay về đúng cuộc trò chuyện. Bản hiện tại chưa tự cuộn tới vị trí của tin được chọn; đây là giới hạn điều hướng cần cải thiện.

9. Triển khai và kịch bản kiểm thử

9.1 Môi trường chạy

Máy chủ Node.js và SQLite chạy bằng Docker Compose; database được giữ trong volume. Android Emulator dùng http://10.0.2.2:8070 để gọi API. Bản Expo Go trên iPhone dùng IP LAN của máy Windows khi hai thiết bị chung Wi-Fi. README.md ghi lệnh cài đặt, build và chạy.

9.2 Kết quả kiểm thử

Bộ test server khởi tạo database tạm và gọi API thật. Test bao phủ xác thực, nhóm/kênh, quyền đọc DM, gửi/reply, sửa/xóa, cảm xúc, tìm kiếm và dữ liệu sau khi khởi động lại server.

9.3 Phương pháp kiểm thử API

Tệp server.test.mjs sử dụng node:test và assert. Trước mỗi lần chạy, test tạo thư mục tạm và file SQLite mới, tìm cổng localhost trống rồi spawn server.mjs với HKCHAT_DB và HKCHAT_PORT riêng. Hàm call gửi HTTP request thật tới server và đọc JSON response. Cách này kiểm tra cả route, xác thực, truy vấn và persistence thay vì chỉ gọi hàm nội bộ.

Một bài test tích hợp bao phủ nhiều assertion liên tiếp. Test tạo năm tài khoản thử, sau đó đi qua tạo nhóm, DM, gửi và reply, sửa, cảm xúc, tìm kiếm, xóa, kênh công khai, phân trang và khởi động lại server. Mỗi assertion đối chiếu status hoặc payload với kết quả mong đợi. Kết thúc bài test, process server bị dừng và thư mục tạm được xóa. Vì vậy số '1/1 bài test' biểu thị một kịch bản tích hợp dài, không phải chỉ một request.

Khi đánh giá kết quả, cần phân biệt API test tự động với kiểm thử giao diện trên thiết bị. API test xác nhận dữ liệu và quyền; ảnh Android chứng minh một số thao tác nhìn thấy được trong emulator; người dùng đã xác nhận prototype Figma có 18/18 màn hình đi tới được. Báo cáo không gộp ba loại bằng chứng thành một phép đo UX hay hiệu năng.

9.4 Ca kiểm thử tin và thread

Test gửi tin 'private hello' trong DM, sau đó so sánh nội dung đọc từ phía thành viên còn lại. Tin mới có replyTo null và mảng reactions rỗng. Bob gửi một tin trả lời; test xác nhận replyTo đúng id tin gốc. Reply tới id không tồn tại bị từ chối 400, còn người ngoài DM cố gửi reply bị từ chối 403.

Test tạo thêm một phản hồi lồng nhau rồi gọi /api/messages/:id/thread. Kết quả trả chuỗi id từ tin gốc qua tin trả lời đến tin lồng. Khi Charlie gọi endpoint thread của DM, API trả 403. Các assertion này kiểm tra cả quan hệ đệ quy và lớp quyền truy cập trước khi dữ liệu thread được trả.

Bob sửa tin trả lời của mình; test xác nhận body đã trim và nội dung cũ không còn xuất hiện trong tìm kiếm. Alice và Charlie không thể sửa tin của Bob, dù Alice là thành viên DM. Nội dung chỉ có khoảng trắng bị từ chối 400. Sau khi Bob xóa tin, sửa lại hoặc gắn cảm xúc vào tin đã xóa đều trả 404; thread và tìm kiếm không chứa tin đó.

9.5 Ca kiểm thử cảm xúc, tìm kiếm và lưu trữ

Với cảm xúc, test thử người ngoài DM và emoji rỗng trước; hai thao tác lần lượt trả 403 và 400. Alice và Bob cùng gắn 👍 lên một tin; số lượt trở thành 2, cờ me trong response của từng người là true. Gắn lại cùng emoji của cùng người không làm tăng count vì khóa chính ghép và INSERT OR IGNORE. Khi Bob bỏ cảm xúc, Alice vẫn thấy count 1 và me true.

Test tạo một kênh công khai mới. Bob chưa tham gia nên chưa đọc được tin; sau POST /join, request đọc trả 200. Kịch bản gửi 105 tin vào kênh này để xác nhận lần đọc đầu chỉ trả 100 tin mới nhất. Với tham số

after trỏ id gần cuối, API chỉ trả tin mới hơn. Đây là kiểm thử phân trang theo id, không phải benchmark số lượng người dùng đồng thời.

Cuối cùng test dừng server và khởi động lại với cùng file SQLite tạm. Tài khoản vẫn đăng nhập được, tin DM và cảm xúc còn tồn tại. Điều này xác nhận dữ liệu được ghi xuống file database, không chỉ giữ trong bộ nhớ process. Trên môi trường Docker, volume đóng vai trò giữ file tương ứng qua vòng đời container.

9.6 Kiểm tra build và giao diện

Lệnh npm.cmd run typecheck chạy TypeScript ở chế độ noEmit để phát hiện lỗi kiểu trong client. Lệnh npm.cmd run test:server chạy bộ test tích hợp nói trên. Android debug build dùng Gradle assembleDebug với JDK 23 trên máy thử; kết quả BUILD SUCCESSFUL đã được ghi nhận. Các phép kiểm tra này cần chạy lại khi mã nguồn thay đổi trước khi nộp bản cuối.

Kiểm thử Android Emulator được minh họa bằng ảnh nhập server/đăng nhập, danh sách kênh, hội thoại nhóm, reply, tìm kiếm và mở lại hội thoại. Ảnh giúp xác nhận bố cục và một số trạng thái chính trên thiết bị ảo. Bản iPhone từ Windows được thử qua Expo Go SDK 58 trên cùng Wi-Fi với server local; người dùng đã đăng nhập thành công khi URL trỏ tới IP LAN đúng. Báo cáo không gọi đó là bản iOS native đã build bằng Xcode.

Trong Figma, hai plugin của dự án tạo màn hình dạng layer và bổ sung Design System cùng liên kết prototype. Người dùng đã xác nhận plugin báo 18/18 màn hình đi tới được từ Flow 1. Kết quả đó kiểm tra đồ thị điều hướng của thiết kế; nó khác với việc xác thực dữ liệu nhập thật trên ứng dụng chạy. Vì vậy phần trình bày giữ tách biệt giữa prototype và ảnh chụp Android.

9.7 Ranh giới kết quả

Những mục có bảng trong migration 002_features.sql như gọi thoại, AI, playbook hoặc push không được báo cáo như tính năng đang sử dụng, vì server hiện không có route nghiệp vụ và client không có màn hình tương ứng cho chúng. Tương tự, bảng attachments trong 001_core.sql không chứng minh tải tệp đã hoạt động. Phạm vi kiểm thử và ảnh minh họa của tài liệu tập trung vào xác thực, kênh, DM, tin nhắn và tìm kiếm.

Ứng dụng local dùng HTTP cho thử nghiệm mạng nội bộ. Khi đưa ra mạng ngoài, cần một cấu hình TLS và chính sách vận hành riêng; báo cáo này không tuyên bố đã có triển khai công khai. Không có số đo tải, thời gian phản hồi hoặc tỷ lệ lỗi trên nhiều thiết bị, nên các chỉ tiêu hiệu năng không được gán giá trị. Cách ghi giới hạn này giúp phân biệt sản phẩm hiện có với những khả năng chỉ mới có cấu trúc dữ liệu hoặc ý tưởng.

9.8 Bảng kịch bản kiểm thử chuẩn hóa

Mỗi ca dưới đây nêu dữ liệu đầu vào và kết quả mong đợi, tương ứng với các assertion trong server.test.mjs. “Đạt” ở đây chỉ có nghĩa các ca tự động đã chạy qua trong môi trường kiểm thử local; chưa suy ra độ ổn định khi tải lớn hoặc trên mọi thiết bị.

ID	Mô tả	Dữ liệu đầu vào	Kết quả mong đợi
TC-01	Đăng ký hợp lệ	Năm tài khoản thử, gồm tên Đạt	201 cho từng tài khoản
TC-02	Nhóm sai thành viên	Danh sách có userId không tồn tại	400; không tạo nhóm
TC-03	Sai mật khẩu	Tài khoản đúng, mật khẩu sai	401; không cấp phiên
TC-04	Không có token	GET /api/channels thiếu Authorization	401; không trả dữ liệu
TC-05	Tạo kênh	Tên kênh hợp lệ	201; người tạo là thành viên

TC-06	Tham gia kênh	POST join kênh public	200; kênh trong danh sách
TC-07	Gửi tin	Body hợp lệ trong kênh đã tham gia	201; đọc lại thấy nội dung
TC-08	Sửa thành tin rỗng	PATCH body chỉ có khoảng trắng	400; không ghi nội dung rỗng
TC-09	Tạo DM	Alice mở DM với Bob	200; trả về channel và id
TC-10	Người ngoài đọc DM	User thứ ba GET messages DM	403; không lộ nội dung
TC-11	Reply sai đích	replyTo = 999999 không tồn tại	400; không ghi phản hồi
TC-12	Reply hợp lệ	replyTo là tin còn tồn tại cùng kênh	201; có liên kết tới tin gốc
TC-13	Sửa tin của người khác	PATCH bằng token người khác	403; nội dung không đổi
TC-14	Xóa tin của người khác	DELETE bằng token người khác	403; tin còn đọc được
TC-15	Cảm xúc trùng	Cùng user thả cùng emoji hai lần	Một reaction; tổng không tăng lặp
TC-16	Tìm kiếm theo quyền	Từ khóa có trong DM và kênh ngoài quyền	Chỉ trả tin được phép xem
TC-17	Dữ liệu sau khởi động lại	Ghi tin rồi khởi động lại server cùng DB	Tin và cảm xúc còn tồn tại

9.9 Kịch bản trình diễn trên Android Emulator

Kịch bản trình diễn bắt đầu bằng việc kiểm tra Docker Desktop đang chạy và GET http://localhost:8070/api/health trả status ok. Trong Android Studio, nhóm mở emulator rồi chạy ứng dụng bằng npm.cmd run android. Tại form đầu, URL phải là http://10.0.2.2:8070. Nếu URL sai, phản hồi lỗi kết nối trước khi thử tài khoản giúp phân biệt mạng và xác thực.

Người trình diễn có thể đăng ký tài khoản mới hoặc dùng tài khoản mẫu trong database của nhóm. Sau đăng nhập, kiểm tra tab Kênh có General, Random và các kênh đã tham gia. Chọn một kênh để xem tin; trở lại danh sách, nhập tên kênh mới và bấm Tạo kênh. Từ tài khoản thứ hai, tìm kênh đó trong Kênh có thể tham gia, bấm Tham gia và gửi một tin. Bước này chứng minh membership được cập nhật qua API.

Sang tab Mọi người, chọn một thành viên để mở DM. Gửi một tin riêng và đăng nhập tài khoản khác trên thiết bị hoặc qua bộ test để xác nhận người ngoài không đọc được. Trong hội thoại nhóm mẫu, nhấn giữ một tin, chọn Trả lời, nhập nội dung và gửi; sau đó thử thả cảm xúc bằng dấu cộng. Với tin của mình, mở tùy chọn để sửa hoặc xóa. Với tin của người khác, API vẫn bảo vệ quyền tác giả dù giao diện có hay không hiển thị nút.

Cuối cùng sang tab Tìm kiếm, nhập một từ có trong hội thoại, xem kết quả và chạm vào tin để mở lại kênh. Màn hiện tại mở hội thoại nhưng không tự cuộn đến đúng tin, nên khi demo cần mô tả đúng giới hạn này. Chọn Đăng xuất để kết thúc phiên; mở lại app nhằm kiểm tra token không còn khôi phục phiên vừa hủy.

9.10 Kịch bản thử trên iPhone cùng mạng

Vì máy phát triển dùng Windows, bản thử iPhone chạy trong Expo Go SDK 58. Trước tiên, Compose cần bind API ra mạng nội bộ bằng HKCHAT_BIND=0.0.0.0 và container được tạo lại. Từ iPhone cùng Wi-Fi, Safari mở http://<IP-LAN-Windows>:8070/api/health. Nếu trang này không trả JSON, lỗi thuộc mạng, firewall hoặc địa chỉ; không nên thử đăng nhập trước khi kết nối cơ bản hoạt động.

Bản Expo Go được chạy từ expo-preview với Metro trên cổng 8082. Người dùng đăng nhập cùng Expo Account trên máy tính và iPhone, quét QR rồi mở bản xem thử. Form của HKchat cần trỏ tới IP LAN của Windows, không phải localhost hay 10.0.2.2. Khi server reachable và URL đúng, các thao tác đăng nhập, mở kênh, DM và gửi tin dùng cùng API Node.js với bản Android.

Kịch bản này đã từng được kiểm tra đến mức màn đăng nhập và đăng nhập thành công trên iPhone trong mạng nội bộ. Báo cáo không suy ra bản iOS native đã ký và cài độc lập. Để tạo IPA hoặc chạy từ Xcode, project ios/HKchat.xcodeproj cần máy Mac và cấu hình signing của Apple.

9.11 Nhật ký lỗi khởi động Docker và cách xác nhận

Một lần chạy mới trên Windows cho thấy npm.cmd ci cài 596 package thành công, nhưng Docker CLI ban đầu báo không tìm thấy pipe dockerDesktopLinuxEngine. Đây là dấu hiệu Docker Desktop/engine chưa chạy, không phải lỗi TypeScript hoặc package của HKchat. Sau khi Docker Desktop khởi động, Compose lần đầu báo cổng 8070 tạm thời không bind được. Kiểm tra netstat không thấy listener bền vững trên 8070 và cổng này không thuộc dải cổng Windows loại trừ.

Chạy lại docker compose -f hkchat/compose.yaml up -d đã khởi động container. docker compose ps hiển thị trạng thái healthy và mapping 127.0.0.1:8070->8070/tcp. Lệnh curl trả {status:ok, product:HKchat}. Chuỗi bằng chứng này phân biệt một lỗi khởi động môi trường tạm thời với lỗi API. Khi tài liệu hướng dẫn máy mới, nhóm đặt kiểm tra Docker Desktop và healthcheck trước lệnh mở app.

10. Ràng buộc và giả định

10.1 Ràng buộc triển khai

Bản server được cấu hình để chạy trên máy của nhóm. Android Emulator gọi host qua 10.0.2.2; iPhone chạy Expo Go dùng IP LAN. Hai thiết bị chỉ trao đổi được khi cùng mạng và server bind đúng địa chỉ. Nếu Docker Desktop chưa chạy hoặc cổng 8070 bị chiếm, app không thể đăng nhập dù client đã build thành công.

Client React Native native có dự án iOS nhưng Windows không thể build IPA bằng Xcode. Bản trên iPhone được kiểm thử qua Expo Go SDK 58, là môi trường xem thử dùng cùng giao diện và API. Đây không phải bằng chứng cho bản iOS đã ký và phát hành. Thông báo đẩy khi app đóng cần cấu hình Firebase/Apple riêng và không được nghiệm thu trong báo cáo này.

10.2 Giả định trong kịch bản thử

- **Server cùng dữ liệu SQLite còn hoạt động.** Tài khoản và hội thoại mẫu hiện được; đổi volume có thể làm mất dữ liệu mẫu.
- **Người kiểm thử có tài khoản thuộc nhóm.** Có quyền đọc và gửi trong hội thoại mẫu.
- **Thiết bị kết nối cùng LAN khi thử iPhone.** URL IP của máy chủ có thể được truy cập từ Safari và Expo Go.
- **Chưa có bài đo tải hoặc nghiên cứu người dùng.** Không đưa ra chỉ tiêu FPS, độ trễ, tải đồng thời hay mức hài lòng.

10.3 Giới hạn chức năng và hướng phát triển

Giao diện hiện chưa có luồng tự tạo nhóm dù API hỗ trợ nhóm; tìm kiếm chỉ theo từ khóa, chưa có bộ lọc nâng cao và chưa tự cuộn đến đúng tin sau khi mở kết quả. App chưa có tải tệp, cuộc gọi hay push khi đã đóng. Các migration có thể chứa bảng cho kế hoạch tương lai nhưng sự tồn tại của bảng không phải bằng chứng chức năng đã hoàn thành.

Nếu tiếp tục phát triển, nhóm nên ưu tiên màn hình tạo/quản lý nhóm, điều hướng tới đúng tin từ kết quả tìm kiếm, phân trang lịch sử, kiểm thử thông báo đẩy và chạy thử trên nhiều thiết bị. Mỗi mục cần có tiêu chí nghiệm thu, ca test và ảnh hoặc log kết quả trước khi đưa vào báo cáo như một chức năng có thật.

11. Kết luận và hướng phát triển

HKchat đã có client di động, API và database local cho luồng nhắn tin nội bộ. Kênh, DM, reply, cảm xúc, sửa/xóa và tìm kiếm hoạt động trên cùng mô hình dữ liệu có kiểm tra quyền thành viên. Thiết kế Figma, ảnh Android và kết quả kiểm thử là căn cứ để nhóm trình bày sản phẩm.

11.1 Bài học kỹ thuật và kết luận

Việc dùng chung channels và messages cho kênh, DM và nhóm giúp thống nhất luồng gửi tin nhưng đòi hỏi kiểm tra membership nhất quán. Trong kiểm thử, khả năng từ chối người ngoài đọc DM và sửa tin của người khác quan trọng ngang với việc gửi tin thành công. Thiết kế database có khóa ngoại, ràng buộc duy nhất và xóa mềm giúp các thao tác này có trạng thái rõ ràng.

Một bài học vận hành là build client thành công không đồng nghĩa API có thể kết nối. Việc kiểm tra Docker engine, cổng 8070 và /api/health phải diễn ra trước khi thử đăng nhập. Trên iPhone, địa chỉ 10.0.2.2 của Android Emulator không có tác dụng; phải nhập IP LAN của máy Windows. Báo cáo ghi lại hai đường chạy này để nhóm khác tái hiện đúng môi trường.

Kết quả của đồ án là một ứng dụng nhắn tin văn bản chạy local với đăng ký, đăng nhập, kênh, DM, hội thoại nhóm mẫu, trả lời, cảm xúc, sửa/xóa và tìm kiếm. Mã nguồn, migration, bộ kiểm thử API, ảnh Android và prototype Figma là các minh chứng cho phạm vi đó. Các giới hạn ở chương trước xác định rõ phần chưa được nghiệm thu và là cơ sở cho vòng phát triển tiếp theo.

Phụ lục thuật ngữ và đối chiếu mã nguồn

Các thuật ngữ dưới đây được dùng nhất quán trong báo cáo để người đọc đối chiếu thiết kế, API và database. Chúng phản ánh cách HKchat đặt tên ở bản hiện tại.

- **Kênh công khai.** *Nghĩa trong HKchat:* Cuộc trò chuyện có thể tìm và tham gia. *Vị trí kiểm tra:* channels.kind = public; /api/channels/public.
- **DM.** *Nghĩa trong HKchat:* Cuộc trò chuyện hai người với dm_key duy nhất. *Vị trí kiểm tra:* POST /api/dm; channel_members.
- **Nhóm.** *Nghĩa trong HKchat:* Channel được đánh dấu trong group_chats. *Vị trí kiểm tra:* POST /api/groups; nhóm mẫu trong DB.
- **Phiên.** *Nghĩa trong HKchat:* Bearer token có hạn, lưu hash trên server. *Vị trí kiểm tra:* sessions; /api/me, /api/logout.
- **Reply.** *Nghĩa trong HKchat:* Tin chứa id tin được trả lời. *Vị trí kiểm tra:* messages.reply_to; thread API.
- **Reaction.** *Nghĩa trong HKchat:* Cảm xúc của một người trên một tin. *Vị trí kiểm tra:* reactions; message DTO.
- **Xóa mềm.** *Nghĩa trong HKchat:* Ghi deleted_at và lọc khỏi kết quả. *Vị trí kiểm tra:* messages.deleted_at.
- **Flow 1.** *Nghĩa trong HKchat:* Điểm bắt đầu prototype Figma. *Vị trí kiểm tra:* Màn 01, 18/18 trạng thái đi tới được.
- **Healthcheck.** *Nghĩa trong HKchat:* Truy vấn kiểm tra API còn đáp ứng. *Vị trí kiểm tra:* GET /api/health; compose.yaml.

Các đường dẫn nêu trong phần này là điểm bắt đầu để kiểm tra lại từng khẳng định. Test API nằm trong hkchat/server/server.test.mjs; giao diện di động trong hkchat/mobile/App.tsx; schema trong hkchat/server/migrations; cấu hình container trong hkchat/compose.yaml; ảnh kiểm thử ở docs/report-images; file Figma được liên kết tại phần thiết kế.

Tài liệu tham khảo

- Slack Channels: https://slack.com/features/channels
- Microsoft Teams channels overview: https://learn.microsoft.com/en-us/microsoftteams/teams-channels-overview
- Google Chat help: https://support.google.com/chat/answer/7659784?hl=en
- Zulip topics: https://zulip.com/help/introduction-to-topics
- Rocket.Chat documentation: https://docs.rocket.chat/docs